

One command, one page — every flag, every response

dnshcli

Command

Reference

Complete reference for all 242 dnshcli operations.
Each page covers one command: what it does, every
flag explained, an example call, and the response you
will receive.

UPDATED

2026-07-13

COMMANDS

242 operations across 36 endpoints

AUDIENCE

Developers & Administrators

Getting Started

dnsfcli is a command-line tool for the complete DNSFilter REST API. Every endpoint and operation is available from the terminal. This reference documents all 242 operations — one per page — with the exact flags, a complete example command, and the response you can expect.

Install

INSTALL

```
pip install -e .
```

Authenticate

Store your API token and default organization ID in the OS keychain once. Every command picks them up automatically.

AUTH SETUP

```
python dnsfcli.py auth setup
python dnsfcli.py auth setup --org-id 802315
python dnsfcli.py auth verify
python dnsfcli.py auth show
```

Global flags

Output & format

<code>--raw, -r</code>	Print the raw JSON response instead of the formatted table.
<code>--json / --jsonl</code>	JSON (or one object per line) on stdout. Automatic when output is piped.
<code>--output FMT</code>	Unified format switch: table, json, jsonl, raw, csv, or none.
<code>--columns a,b,c</code>	Limit table/CSV output to the named columns.
<code>--preset NAME</code>	Apply a named column preset from config (column_presets.NAME).
<code>--format TMPL</code>	Render each row through a template — both "{name} ({id})" and "{[.name]}" styles work.
<code>--format-preset N</code>	Apply a named format template from config (format_presets.NAME).

<code>--bundle NAME</code>	Apply a named command bundle from config (columns + filter + sort + format).
<code>--pick FIELD</code>	Print a single field, one value per line — ideal for piping.
<code>--jq PATH</code>	Extract a dotted path (data.0.name) from the response before output.
<code>--to-csv FILE</code>	Write the response to FILE as CSV (use - for stdout).
<code>--to-json FILE</code>	Write the response to FILE as JSON.
<code>--to-markdown FILE</code>	Write the response to FILE as a Markdown table (use - for stdout).
<code>--tee FILE</code>	Save a copy of everything printed to FILE.
<code>--quiet, -q</code>	Suppress status chatter; result data still prints.
<code>--no-color</code>	Disable ANSI colour output (NO_COLOR env also honoured).

Filtering, sorting & aggregation

<code>--filter EXPR</code>	Keep matching rows: field=value, field!=value, field~substr, field>N, >=, <, <=.
<code>--filter-mode or</code>	OR logic across multiple <code>--filter</code> flags (default: AND).
<code>--grep REGEX</code>	Keep rows where any field matches the regex.
<code>--unique FIELD</code>	Drop rows with a duplicate value in FIELD.
<code>--sort FIELD</code>	Sort by field; prefix with - for descending (<code>--sort -created_at</code>).
<code>--limit N / --last N</code>	Return at most N results / the last N results.
<code>--count</code>	Print the result count only.
<code>--count-by FIELD</code>	Frequency table of FIELD values.
<code>--group-by FIELD</code>	Group results by FIELD value.
<code>--sum / --avg / --min / --max F</code>	Aggregate a numeric field and print one value.
<code>--select a,b / --exclude a,b</code>	Keep only (or remove) the named fields from every item.
<code>--not-null F / --is-null F</code>	Keep rows where FIELD is (or is not) null.

Pagination

<code>--all</code>	Fetch every page of paginated results.
<code>--page N / --page-size N</code>	Request a specific page / page size.
<code>--max-pages N</code>	Cap the number of pages fetched by <code>--all</code> .
<code>--paginate-until E</code>	Stop <code>--all</code> pagination once any item matches filter expression E.

Batch & CSV input

<code>--from-csv FILE</code>	Read input rows from FILE — one API call per row (use - for stdin).
<code>--template</code>	Print a blank CSV import template for this command and exit. No auth needed.
<code>--skip-rows N / --max-rows N</code>	Resume an interrupted batch / cap rows processed.
<code>--batch-report FILE</code>	Write a JSON run summary (per-row outcomes, counts) to FILE.
<code>--on-error MODE</code>	stop or continue when a batch row fails.
<code>--errors-to-csv FILE</code>	Write failed rows to FILE for later retry (<code>--retry-errors-csv</code>).
<code>--confirm-each</code>	Prompt before each batch row.

<code>--validate-only</code>	Validate the CSV and exit without calling the API.
<code>--dry-run / --plan</code>	Show what would be sent without making any API calls.
<code>--yes, -y</code>	Skip confirmation prompts.

Transformation

<code>--add-field K=V</code>	Inject a static field into every result item.
<code>--transform F=EXPR</code>	Compute a new field from a restricted expression (ratio=blocked/total).
<code>--map FIELD=OP</code>	Transform a field value: upper, lower, strip, title, truncate:N.
<code>--join EP:LK=RK</code>	Client-side join: fetch endpoint EP, match local key LK to remote key RK.
<code>--rename A=B</code>	Rename field A to B in the output.
<code>--flatten</code>	Flatten nested objects into dotted keys.
<code>--strip-nulls</code>	Remove null-valued keys from every item.

Multi-organization

<code>--each-org</code>	Run the command once per organization on the account.
<code>--org-csv FILE</code>	Load the <code>--each-org</code> organization list from a CSV file.
<code>--org-filter REGEX</code>	Restrict <code>--each-org</code> to organizations whose name matches.
<code>--max-orgs N</code>	Cap the number of organizations processed.
<code>--parallel-orgs</code>	Process organizations concurrently (<code>--org-concurrency N</code>).

Watch, monitor & CI

<code>--watch N</code>	Re-run the command every N seconds (Ctrl-C to stop).
<code>--watch-until EXPR</code>	Stop watching once any result matches the filter expression.
<code>--watch-diff</code>	Show only what changed between watch ticks.
<code>--alert EXPR</code>	Ring the terminal bell and print a banner when a result matches.
<code>--fail-on-empty</code>	Exit 1 when the result list is empty (messages go to stderr).
<code>--fail-on-pattern E</code>	Exit 1 when any result row matches filter expression E.
<code>--exec CMD</code>	Run a shell command per result row with {field} substitution (values are shell-quoted).

Connection & authentication

<code>--api-key TOKEN</code>	Override the keychain token for this call only (prefer DNSF_API_KEY).
<code>--org-id ID</code>	Override the stored organization ID for this call only.
<code>--profile NAME</code>	Use a named credential profile.
<code>--timeout N / --connect-timeout N</code>	Read / connect timeouts in seconds.
<code>--rate N</code>	Client-side request-per-second cap.
<code>--retry N</code>	Retry attempts for failed batch rows.
<code>--header K=V</code>	Add a custom request header (scrubbed from history logs).
<code>--proxy URL</code>	Route requests through a proxy (scrubbed from history logs).

<code>--env-file FILE</code>	Load KEY=VALUE pairs into the environment before running.
<code>--cache-ttl N</code>	Serve identical GETs from a local cache for N seconds.

agent-local-users bulk-delete

POST /v1/agent_local_users_bulk_delete

Bulk delete agent local users.

FLAGS

--ids [required] An array of resource IDs. Example: ["id1","id2"], e.g. --ids '["1001","1002","1003"]'
--exclude_ids An array of resource IDs to exclude from the operation, e.g. --exclude_ids ["other-agent-uuid"]

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users bulk-delete --ids '["1001","1002","1003"]'
```

RESPONSE

HTTP 202 Accepted

```
{
  "id": "job-alu-bd-001",
  "status": "pending",
  "ids": [
    1001,
    1002,
```

1003

]

}

agent-local-users bulk-delete-counts

GET /v1/agent_local_users_bulk_delete/counts

Count agent local users matching bulk-delete criteria.

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users bulk-delete-counts
```

RESPONSE

HTTP 200 OK

```
{
  "count": 47,
  "ids": [
    1001,
    1002,
    1003,
    {
      "...": "44 more"
    }
  ]
}
```


agent-local-users bulk-delete-show

GET /v1/agent_local_users_bulk_delete/{id}

Show a bulk-delete job.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id job-alu-bd-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users bulk-delete-show --id job-alu-bd-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-alu-bd-001",
  "status": "completed",
  "ids": [
    1001,
    1002,
    1003
  ],
  "deleted_count": 3,
```

```
"created_at": "2025-06-01T10:00:00.000-04:00",  
"completed_at": "2025-06-01T10:00:05.000-04:00"  
}
```

agent-local-users delete

DELETE /v1/agent_local_users/{id}

Permanently delete a agent-local-users resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 1001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users delete --id 1001
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

agent-local-users list

GET /v1/agent_local_users

Retrieve a paginated list of agent-local-users resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "4439029",
      "type": "agent_local_users",
      "uuid": "0195f32d-d61e-7d6b-844f-d36555270d9a",
      "attributes": {
        "friendly_name": null,
```

```
"user_login": "SIGOFFICEUPD\\tv",
"user_name": "tv",
"user_remote_id": "S-1-5-21-931150416-2819136805-1750517711-1001",
"first_seen": "2025-04-01T17:07:02.294-04:00",
"last_seen": "2025-04-04T16:46:33.781-04:00",
"in_a_collection": true,
"collections": [
  {
    "id": 15345,
    "name": "CLI Test",
    "organization_id": 802315,
    "deleted_at": null,
    "created_at": "2026-05-20T12:33:07.748-04:00",
    "updated_at": "2026-05-20T12:33:30.520-04:00",
    "policy_id": 331207,
    "scheduled_policy_id": null,
    "block_page_id": null,
    "order": 1,
    "is_manual": true,
    "sync_tool_collection_ids": [],
    "sync_tool_group_ids": [],
    "uuid": "019e463b-d8a4-799b-98e5-37628466e3a3",
    "description": null
  }
],
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  },
  "policy": {
    "data": null
  },
  "scheduled_policy": {
    "data": null
  },
  "block_page": {
```

```
    "data": null
  }
},
{
  "id": "4042678",
  "type": "agent_local_users",
  "uuid": "01944bc6-091c-7bc8-8589-5d6d90e4ab16",
  "attributes": {
    "friendly_name": null,
    "user_login": "DESKTOP-01\\jsmith",
    ...
  }
}
```

agent-local-users list-all

GET /v1/agent_local_users/all

Retrieve all agent-local-users resources without pagination limits.

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "4439029",
      "type": "agent_local_users",
      "uuid": "0195f32d-d61e-7d6b-844f-d36555270d9a",
      "attributes": {
        "friendly_name": null,
        "user_login": "SIGOFFICEUPD\\tv",
        "user_name": "tv",
        "user_remote_id": "S-1-5-21-931150416-2819136805-1750517711-1001",
        "first_seen": "2025-04-01T17:07:02.294-04:00",
        "last_seen": "2025-04-04T16:46:33.781-04:00",
        "in_a_collection": true,
        "collections": [
          {
            "id": 15345,
            "name": "CLI Test",
            "organization_id": 802315,
            "deleted_at": null,
            "created_at": "2026-05-20T12:33:07.748-04:00",
            "updated_at": "2026-05-20T12:33:30.520-04:00",
            "policy_id": 331207,
            "scheduled_policy_id": null,
            "block_page_id": null,

```

```

    "order": 1,
    "is_manual": true,
    "sync_tool_collection_ids": [],
    "sync_tool_group_ids": [],
    "uuid": "019e463b-d8a4-799b-98e5-37628466e3a3",
    "description": null
  }
]
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  },
  "policy": {
    "data": null
  },
  "scheduled_policy": {
    "data": null
  },
  "block_page": {
    "data": null
  }
}
},
{
  "id": "4042678",
  "type": "agent_local_users",
  "uuid": "01944bc6-091c-7bc8-8589-5d6d90e4ab16",
  "attributes": {
    "friendly_name": null,
    "user_login": "DESKTOP-01\\jsmith",
    ...

```


agent-local-users show

GET /v1/agent_local_users/{id}

Retrieve a single agent-local-users resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 1001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py agent-local-users show --id 1001
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "4439029",
    "type": "agent_local_users",
    "uuid": "0195f32d-d61e-7d6b-844f-d36555270d9a",
    "attributes": {
      "friendly_name": null,
      "user_login": "SIGOFFICEUPD\\tv",
      "user_name": "tv",
```

```

"user_remote_id": "S-1-5-21-931150416-2819136805-1750517711-1001",
"first_seen": "2025-04-01T17:07:02.294-04:00",
"last_seen": "2025-04-04T16:46:33.781-04:00",
"in_a_collection": true,
"collections": [
  {
    "id": 15345,
    "name": "CLI Test",
    "organization_id": 802315,
    "deleted_at": null,
    "created_at": "2026-05-20T12:33:07.748-04:00",
    "updated_at": "2026-05-20T12:33:30.520-04:00",
    "policy_id": 331207,
    "scheduled_policy_id": null,
    "block_page_id": null,
    "order": 1,
    "is_manual": true,
    "sync_tool_collection_ids": [],
    "sync_tool_group_ids": [],
    "uuid": "019e463b-d8a4-799b-98e5-37628466e3a3",
    "description": null
  }
],
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  },
  "policy": {
    "data": null
  },
  "scheduled_policy": {
    "data": null
  },
  "block_page": {
    "data": null
  }
}

```

```
}  
}  
}
```

agent-local-users update

PATCH /v1/agent_local_users/{id}

Update an existing agent-local-users resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 1001`
`--friendly_name` The display name shown for this agent in the dashboard, e.g. `--friendly_name "Jane Smith Laptop"`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnscli.py agent-local-users update \  
  --id 1001 \  
  --friendly_name "Jane Smith Laptop" \  
  --policy_id 285109
```

RESPONSE

HTTP 200 OK

```
{
  "id": 1001,
  "type": "agent_local_users",
  "attributes": {
    "friendly_name": "Jane Smith Laptop",
    "policy_id": 285109,
    "updated_at": "2025-06-01T10:00:00.000-04:00"
  }
}
```

api-keys create

POST /v1/api_keys

Create a new api-keys resource.

FLAGS

`--name` [required] A human-readable display name for this resource, e.g. `--name "CI Pipeline Key"`
`--expiry` The expiry date in YYYY-MM-DD format. Maximum 1 year from today, e.g. `--expiry 2027-05-31`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py api-keys create --name "CI Pipeline Key" --expiry 2027-05-31
```

RESPONSE

HTTP 201 Created

```
{
  "id": 20982,
  "type": "api_keys",
  "attributes": {
    "name": "CI Pipeline Key",
    "expiry": "2027-05-31",
```

```
"token": "eyJhbGciOiJIUzI1NiJ9.new_key_token_here",  
"created_at": "2025-06-01T10:00:00.000-04:00"  
}  
}
```

api-keys delete

DELETE /v1/api_keys/{id}

Permanently delete a api-keys resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 1618

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py api-keys delete --id 1618
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

api-keys list

GET /v1/api_keys

Retrieve a paginated list of api-keys resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py api-keys list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1618",
      "type": "api_keys",
      "uuid": "019a131c-9218-7fbe-a088-59d79fb9a53c",
      "attributes": {
        "name": "CidrTest",
```

```

    "expiry": "2026-04-24T01:59:59.999-04:00",
    "last_four": "R9Nc",
    "token": "*****R9Nc",
    "status": "active",
    "created_at": "2025-10-23T18:07:10.872-04:00",
    "updated_at": "2025-10-23T18:07:10.872-04:00",
    "user_id": 42618
  }
},
{
  "id": "180",
  "type": "api_keys",
  "uuid": "01944cdc-286d-79cb-93ff-6599c949f340",
  "attributes": {
    "name": "UHMTest",
    "expiry": "2025-03-10T15:57:57.382-04:00",
    "last_four": "DM08",
    "token": "*****DM08",
    "status": "active",
    "created_at": "2025-01-09T15:58:04.269-05:00",
    "updated_at": "2025-01-09T15:58:04.269-05:00",
    "user_id": 42618
  }
},
{
  "id": "2864",
  "type": "api_keys",
  "uuid": "019e1ced-c85f-7c6b-a328-1cfbe3e247af",
  "attributes": {
    "name": "IPScriptTest",
    "expiry": "2027-05-13T01:59:59.999-04:00",
    "last_four": "k3Qw",
    "token": "*****k3Qw",
    "status": "active",
    "created_at": "2026-05-12T12:03:25.918-04:00",
    "updated_at": "2026-05-12T12:03:25.918-04:00",
    "user_id": 42618
  }
}
]
}

```

api-keys revoke

POST /v1/api_keys/{id}/revoke

Revoke an API key.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 1618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py api-keys revoke --id 1618
```

RESPONSE

HTTP 200 OK

```
{
  "id": 20981,
  "type": "api_keys",
  "attributes": {
    "name": "Old Key",
    "revoked_at": "2025-06-01T10:00:00.000-04:00",
    "status": "revoked"
  }
}
```

```
}  
}
```

api-keys show

GET /v1/api_keys/{id}

Retrieve a single api-keys resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 1618

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py api-keys show --id 1618
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "1618",
    "type": "api_keys",
    "uuid": "019a131c-9218-7fbe-a088-59d79fb9a53c",
    "attributes": {
      "name": "CidrTest",
      "expiry": "2026-04-24T01:59:59.999-04:00",
      "last_four": "R9Nc",
```

```
"token": "*****R9Nc",  
"status": "active",  
"created_at": "2025-10-23T18:07:10.872-04:00",  
"updated_at": "2025-10-23T18:07:10.872-04:00",  
"user_id": 42618  
}  
}  
}
```

application-categories list

GET /v1/application_categories

Retrieve a paginated list of application-categories resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py application-categories list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1016",
      "type": "application_categories",
      "uuid": "0199e92d-334d-797b-a335-ce0fbcb28ce9",
      "attributes": {
        "name": "BETA",
```

```
    "description": null
  }
},
{
  "id": "2",
  "type": "application_categories",
  "uuid": "017d0b30-0ecf-7046-8da6-f69a63571fdd",
  "attributes": {
    "name": "Business",
    "description": null
  }
},
{
  "id": "3",
  "type": "application_categories",
  "uuid": "017d0b30-0ed6-7a35-a293-c6d66ab9f559",
  "attributes": {
    "name": "Cybersecurity",
    "description": null
  }
},
{
  "id": "13",
  "type": "application_categories",
  "uuid": "017d0b30-7967-75ca-8626-ac09ade87bb0",
  "attributes": {
    "name": "Ecosystem Applications",
    "description": null
  }
},
{
  "id": "4",
  "type": "application_categories",
  "uuid": "017d0b30-0edd-775f-89fe-da92522f0b1f",
  "attributes": {
    "name": "File Sharing",
    "description": null
  }
},
{
  "id": "5",
```



```
"type": "application_categories",
"uuid": "017d0b30-0ee3-7932-837a-bbb6c69fb824",
"attributes": {
  "name": "Financial",
  "description": null
}
},
{
  "id": "1015",
  "type": "application_categories",
  "uuid": "01954d3b-163f-7919-900b-3dfe6903ab3",
  ...
```

application-categories show

GET /v1/application_categories/{id}

Retrieve a single application-categories resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 1

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py application-categories show --id 1
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "1",
    "type": "application_categories",
    "uuid": "017d0b30-0ec7-7e20-91ad-6380f089a6d8",
    "attributes": {
      "name": "Uncategorized",
      "description": null
    }
  }
}
```

```
}  
}
```

applications list

GET /v1/applications

Retrieve a paginated list of applications resources.

EXAMPLE COMMAND

```
python dnscli.py applications list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "496",
      "type": "applications",
      "uuid": "0179c86c-0992-7d61-9975-b1b5b99e3003",
      "attributes": {
        "name": "3cx",
        "display_name": "3CX",
        "description": "3CX provides easy to use IP PBX solutions. The solution can run on-
premise or in the cloud.",
        "home_page_url": "https://www.3cx.com",
        "favicon": "https://static.netify.ai/logos/3/c/x/3pk/favicon.ico?v=2",
        "icon": "https://static.netify.ai/logos/3/c/x/3pk/icon.png?v=2",
        "logo": "https://static.netify.ai/logos/3/c/x/3pk/logo.png?v=2",
        "dnsfilter_live_app": true,
        "created_at": "2021-06-01T12:33:14.130-04:00"
      },
      "relationships": {
        "application_category": {
          "data": {
            "id": 11,
            "name": "VoIP/Conferencing",
            "description": null
          }
        }
      }
    }
  ]
}
```

```

    }
  }
},
{
  "id": "479",
  "type": "applications",
  "uuid": "0179c86c-0694-7ea3-9351-b51a4010634c",
  "attributes": {
    "name": "4shared",
    "display_name": "4shared",
    "description": "4shared is an online storage and file hosting web service to upload,
store and download music, videos, photographs and other content.",
    "home_page_url": "https://www.4shared.com",
    "favicon": "https://static.netify.ai/logos/4/s/h/4funerq/favicon.ico?v=3",
    "icon": "https://static.netify.ai/logos/4/s/h/4funerq/icon.png?v=3",
    "logo": "https://static.netify.ai/logos/4/s/h/4funerq/logo.png?v=3",
    "dnsfilter_live_app": true,
    "created_at": "2021-06-01T12:33:13.364-04:00"
  },
  "relationships": {
    "application_category": {
      "data": {
        "id": 4,
        "name": "File Sharing",
        "description": null
      }
    }
  }
},
{
  "id": "105",
  "type": "applications",
  "uuid": "0179c86b-c914-793f-a9c8-7acbf7f9d368",
  "attributes": {
    "name": "amazon",
    "display_name": "Amazon",
    "description": "Amazon is an e-commerce, shipping, cloud computing, and digital
streaming provider. They are one of the largest online shopping companies in the world.",
    ...

```

applications list-all

GET /v1/applications/all

Retrieve all applications resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py applications list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "9",
      "type": "applications",
      "uuid": "0179c86b-b984-75df-929b-b2c90867603f",
      "attributes": {
        "name": "101domains",
        "display_name": "101domains",
        "description": "101domain is an established domain management company with experience in providing international domain name solutions for tens of thousands of clients.",
        "home_page_url": "https://www.101domain.com",
        "favicon": "https://static.netify.ai/logos/1/0/1/101qbznva/favicon.png?v=2",
        "icon": "https://static.netify.ai/logos/1/0/1/101qbznva/icon.png?v=2",
        "logo": "https://static.netify.ai/logos/1/0/1/101qbznva/logo.png?v=2",
        "dnsfilter_live_app": false,
        "created_at": "2021-06-01T12:32:53.636-04:00"
      },
      "relationships": {
        "application_category": {
          "data": {
            "id": 1,
            "name": "Uncategorized",
            "description": null
          }
        }
      }
    }
  ]
}
```

```

    }
  }
},
{
  "id": "1342",
  "type": "applications",
  "uuid": "0186eb8c-bf6e-789b-a5d7-4ec884755186",
  "attributes": {
    "name": "15below",
    "display_name": "15below",
    "description": "15below specializes in passenger communications for the travel
industry. The company provides airlines, rail, and other travel companies the technology to
deliver targeted, personalized and automated notifications to their customers,",
    "home_page_url": "https://15below.com",
    "favicon": "https://static.netify.ai/logos/1/5/b/15orybj/favicon.png?v=1",
    "icon": "https://static.netify.ai/logos/1/5/b/15orybj/icon.ico?v=1",
    "logo": null,
    "dnsfilter_live_app": false,
    "created_at": "2023-03-16T13:51:55.246-04:00"
  },
  "relationships": {
    "application_category": {
      "data": {
        "id": 1,
        "name": "Uncategorized",
        "description": null
      }
    }
  }
},
{
  "id": "390",
  "type": "applications",
  "uuid": "0179c86b-f84b-77c5-9c53-88989bf7e8ac",
  "attributes": {
    "name": "1password",
    "display_name": "1Password",
    "description": "A password manager, digital vault, form filler and secure digital wallet.
1Password remembers all your passwords for you to help keep account information safe.",
    ...

```

applications show

GET /v1/applications/{id}

Retrieve a single applications resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 496

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py applications show --id 496
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "496",
    "type": "applications",
    "uuid": "0179c86c-0992-7d61-9975-b1b5b99e3003",
    "attributes": {
      "name": "3cx",
      "display_name": "3CX",
      "description": "3CX provides easy to use IP PBX solutions. The solution can run on-
```



```
premise or in the cloud.",
  "home_page_url": "https://www.3cx.com",
  "favicon": "https://static.netify.ai/logos/3/c/x/3pk/favicon.ico?v=2",
  "icon": "https://static.netify.ai/logos/3/c/x/3pk/icon.png?v=2",
  "logo": "https://static.netify.ai/logos/3/c/x/3pk/logo.png?v=2",
  "dnsfilter_live_app": true,
  "created_at": "2021-06-01T12:33:14.130-04:00"
},
"relationships": {
  "application_category": {
    "data": {
      "id": 11,
      "name": "VoIP/Conferencing",
      "description": null
    }
  }
}
}
```

billing create

POST /v1/billing

Create a new billing resource.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

`--payment_token` [required] Payment token, e.g. `--payment_token tok_visa_4242`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py billing create --organization_id 802315 --payment_token tok_visa_4242
```

RESPONSE

HTTP 201 Created

```
{
  "id": 802315,
  "type": "billing",
  "attributes": {
    "organization_id": 802315,
    "status": "active"
  }
}
```

```
}  
}
```

billing get-address

GET /v1/billing/address/{organization_id}

Get billing address.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py billing get-address --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "first_name": null,
  "last_name": null,
  "email": null,
  "company": null,
  "phone": null,
  "line1": null,
  "line2": null,
```

```
"line3": null,  
"city": null,  
"state_code": null,  
"state": null,  
"zip": null,  
"country": null  
}
```

billing show

GET /v1/billing

Retrieve a single billing resource by its ID.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py billing show
```

RESPONSE

HTTP 200 OK

Empty response — this is the expected behaviour for this endpoint.

It indicates the operation succeeded but there is no data to return.

```
{}
```

billing update-address

PATCH /v1/billing/address/{organization_id}

Update billing address.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--first_name` The user's given (first) name, e.g. `--first_name "Jane"`
`--last_name` The user's family (last) name, e.g. `--last_name "Smith"`
`--email` An email address. Example: `admin@company.com`, e.g. `--email admin@company.com`
`--company` Company name, e.g. `--company "Acme Corp"`
`--phone` A phone number. Example: `+12025551234`, e.g. `--phone +12025551234`
`--line1` Address line 1, e.g. `--line1 "123 Main St"`
`--line2` Address line 2, e.g. `--line2 "Suite 400"`
`--line3` Address line 3, e.g. `--line3 "example"`
`--city` City, e.g. `--city "Denver"`
`--state` State name, e.g. `--state "Colorado"`
`--state_code` State code (e.g. CO), e.g. `--state_code "CO"`
`--zip` ZIP / postal code, e.g. `--zip 80202`
`--country` Country, e.g. `--country "US"`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnscli.py billing update-address \  
  --organization_id 802315 \  
  --first_name "Jane" \  
  --last_name "Smith" \  
  --line1 "123 Main St" \  
  --city "Denver" \  
  --state "Colorado" \  
  --zip 80202 \  
  --country "US"
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 802315,  
  "type": "billing_address",  
  "attributes": {  
    "first_name": "Jane",  
    "last_name": "Smith",  
    "line1": "123 Main St",  
    "city": "Denver",  
    "state": "Colorado",  
    "zip": "80202",  
    "country": "US"  
  }  
}
```


block-pages create

POST /v1/block_pages

Create a new block-pages resource.

FLAGS

`--name` [required] A human-readable display name for this resource, e.g. `--name "Corporate Block Page"`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--block_org_name` Organization name to display on block page, e.g. `--block_org_name "Acme Corp"`
`--block_email_addr` Contact email shown on block page, e.g. `--block_email_addr admin@company.com`
`--block_logo_uuid` UUID of logo to display, e.g. `--block_logo_uuid uuid-logo-1234`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py block-pages create \  
  --name "Corporate Block Page" \  
  --block_org_name "Acme Corp" \  
  --block_email_addr admin@company.com
```

RESPONSE

HTTP 201 Created

```
{
  "id": 9901,
  "type": "block_pages",
  "attributes": {
    "name": "Corporate Block Page",
    "block_org_name": "Acme Corp",
    "block_email_addr": "admin@company.com"
  }
}
```

block-pages delete

DELETE /v1/block_pages/{id}

Permanently delete a block-pages resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 57840`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py block-pages delete --id 57840
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

block-pages list

GET /v1/block_pages

Retrieve a paginated list of block-pages resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py block-pages list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "57840",
      "type": "block_pages",
      "uuid": "0194cd45-e901-7058-8d83-0e0705474fc5",
      "attributes": {
        "name": "Redirect Test",
```

```

    "organization_id": 802315,
    "block_org_name": "asdasd",
    "block_email_addr": "a@b.com",
    "block_logo_uuid": "https://cdn.dnsfilter.com/10a71ac6-
b52e-4581-9596-62d87897f23a/",
    "block_redirect_url": "https://choctop.us/?",
    "created_at": "2025-02-03T14:24:58.497-05:00",
    "updated_at": "2025-09-11T13:13:31.686-04:00",
    "is_global": false,
    "can_edit": true
  },
  "relationships": {
    "organization": {
      "data": {
        "id": "802315",
        "type": "organizations",
        "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
      }
    },
    "networks": {
      "data": []
    },
    "mac_addresses": {
      "data": []
    },
    "network_subnets": {
      "data": []
    },
    "user_agents": {
      "data": []
    },
    "collections": {
      "data": []
    },
    "agent_local_users": {
      "data": []
    }
  }
},
"links": {

```

```
"self": "https://api.dnsfilter.com/v1/block_pages?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",  
"first": "https://api.dnsfilter.com/v1/block_pages?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",  
"prev": null,  
"next": null,  
"last": "https://api.dnsfilter.com/v1/block_pages?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2"  
}  
}
```

block-pages list-all

GET /v1/block_pages/all

Retrieve all block-pages resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py block-pages list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "57840",
      "type": "block_pages",
      "uuid": "0194cd45-e901-7058-8d83-0e0705474fc5",
      "attributes": {
        "name": "Redirect Test",
        "organization_id": 802315,
        "block_org_name": "asdasd",
        "block_email_addr": "a@b.com",
        "block_logo_uuid": "https://cdn.dnsfilter.com/10a71ac6-b52e-4581-9596-62d87897f23a/",
        "block_redirect_url": "https://choctop.us/",
        "created_at": "2025-02-03T14:24:58.497-05:00",
        "updated_at": "2025-09-11T13:13:31.686-04:00",
        "is_global": false,
        "can_edit": true
      },
      "relationships": {
        "organization": {
          "data": {
            "id": "802315",
            "type": "organizations",
            "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
          }
        }
      }
    }
  ]
}
```

```

    }
  },
  "networks": {
    "data": []
  },
  "mac_addresses": {
    "data": []
  },
  "network_subnets": {
    "data": []
  },
  "user_agents": {
    "data": []
  },
  "collections": {
    "data": []
  },
  "agent_local_users": {
    "data": []
  }
}
],
"links": {
  "self": "https://api.dnsfilter.com/v1/block_pages/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
  "first": "https://api.dnsfilter.com/v1/block_pages/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
  "prev": null,
  "next": null,
  "last": "https://api.dnsfilter.com/v1/block_pages/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500"
}
}

```


block-pages show

GET /v1/block_pages/{id}

Retrieve a single block-pages resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 57840

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py block-pages show --id 57840
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "57840",
    "type": "block_pages",
    "uuid": "0194cd45-e901-7058-8d83-0e0705474fc5",
    "attributes": {
      "name": "Redirect Test",
      "organization_id": 802315,
      "block_org_name": "asdasd",
```

```
"block_email_addr": "a@b.com",
"block_logo_uuid": "https://cdn.dnsfilter.com/10a71ac6-
b52e-4581-9596-62d87897f23a/",
"block_redirect_url": "https://choctop.us/?",
"created_at": "2025-02-03T14:24:58.497-05:00",
"updated_at": "2025-09-11T13:13:31.686-04:00",
"is_global": false,
"can_edit": true
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  },
  "networks": {
    "data": []
  },
  "mac_addresses": {
    "data": []
  },
  "network_subnets": {
    "data": []
  },
  "user_agents": {
    "data": []
  },
  "collections": {
    "data": []
  },
  "agent_local_users": {
    "data": []
  }
}
}
```

block-pages update

PATCH /v1/block_pages/{id}

Update an existing block-pages resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 5147`
`--name` [required] A human-readable display name for this resource, e.g. `--name "Corporate Block Page Updated"`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--block_org_name` Organization name to display on block page, e.g. `--block_org_name "Acme Corp"`
`--block_email_addr` Contact email shown on block page, e.g. `--block_email_addr admin@company.com`
`--block_logo_uuid` UUID of logo to display, e.g. `--block_logo_uuid uuid-logo-1234`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py block-pages update --id 5147 --name "Corporate Block Page Updated"
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 9901,  
  "type": "block_pages",  
  "attributes": {  
    "name": "Corporate Block Page Updated"  
  }  
}
```

categories list

GET /v1/categories

Retrieve a paginated list of categories resources.

EXAMPLE COMMAND

```
python dnscli.py categories list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1",
      "type": "categories",
      "attributes": {
        "source": "webshrinker",
        "name": "Abortion",
        "description": "Sites which provide views either in favor or against abortion, provide details on procedures, offer help or discuss outcomes or consequences of abortion.",
        "security": false,
        "internal": false,
        "webshrinker_name": "abortion"
      },
      "relationships": {
        "child_categories": {
          "data": []
        },
        "parent_categories": {
          "data": []
        }
      }
    },
    {
      "id": "2",
```

```

    "type": "categories",
    "attributes": {
      "source": "webshrinker",
      "name": "Adult Content",
      "description": "Sites which may contain sexually explicit content, images, or that are
portrayed through visually expressive language.",
      "security": false,
      "internal": false,
      "webshrinker_name": "adult"
    },
    "relationships": {
      "child_categories": {
        "data": []
      },
      "parent_categories": {
        "data": []
      }
    }
  },
  {
    "id": "3",
    "type": "categories",
    "attributes": {
      "source": "webshrinker",
      "name": "Advertising",
      "description": "Sites or businesses which directly sell ads to consumer through
various mediums - including Internet, TV, or radio.",
      "security": false,
      "internal": false,
      "webshrinker_name": "advertising"
    },
    "relationships": {
      "child_categories": {
        "data": []
      },
      "parent_categories": {
        "data": []
      }
    }
  }
  ...

```

categories list-all

GET /v1/categories/all

Retrieve all categories resources without pagination limits.

EXAMPLE COMMAND

```
python dnsfcli.py categories list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1",
      "type": "categories",
      "attributes": {
        "source": "webshrinker",
        "name": "Abortion",
        "description": "Sites which provide views either in favor or against abortion, provide details on procedures, offer help or discuss outcomes or consequences of abortion.",
        "security": false,
        "internal": false,
        "webshrinker_name": "abortion"
      },
      "relationships": {
        "child_categories": {
          "data": []
        },
        "parent_categories": {
          "data": []
        }
      }
    },
    {
      "id": "2",
```

```

    "type": "categories",
    "attributes": {
      "source": "webshrinker",
      "name": "Adult Content",
      "description": "Sites which may contain sexually explicit content, images, or that are
portrayed through visually expressive language.",
      "security": false,
      "internal": false,
      "webshrinker_name": "adult"
    },
    "relationships": {
      "child_categories": {
        "data": []
      },
      "parent_categories": {
        "data": []
      }
    }
  },
  {
    "id": "3",
    "type": "categories",
    "attributes": {
      "source": "webshrinker",
      "name": "Advertising",
      "description": "Sites or businesses which directly sell ads to consumer through
various mediums - including Internet, TV, or radio.",
      "security": false,
      "internal": false,
      "webshrinker_name": "advertising"
    },
    "relationships": {
      "child_categories": {
        "data": []
      },
      "parent_categories": {
        "data": []
      }
    }
  }
  ...

```


categories show

GET /v1/categories/{id}

Retrieve a single categories resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 2

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py categories show --id 2
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "2",
    "type": "categories",
    "attributes": {
      "source": "webshrinker",
      "name": "Adult Content",
      "description": "Sites which may contain sexually explicit content, images, or that are portrayed through visually expressive language.",
    }
  }
}
```

```
"security": false,  
"internal": false,  
"webshrinker_name": "adult"  
},  
"relationships": {  
  "child_categories": {  
    "data": []  
  },  
  "parent_categories": {  
    "data": []  
  }  
}  
}  
}
```

collections users-add

POST /v1/collections/{collection_id}/users

Add a user to a collection.

FLAGS

`--collection_id` [required] The numeric ID of the user collection, e.g. `--collection_id 7788`
`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 42618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py collections users-add --collection_id 7788 --id 42618
```

RESPONSE

HTTP 201 Created

```
{
  "id": 42618,
  "type": "users",
  "attributes": {
    "email": "admin@company.com",
    "added_at": "2025-06-01T10:00:00.000-04:00"
  }
}
```

```
}  
}
```

collections users-list

GET /v1/collections/{collection_id}/users

List users in a collection.

FLAGS

--collection_id [required] The numeric ID of the user collection, e.g. --collection_id 7788

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py collections users-list --collection_id 7788
```

RESPONSE

HTTP 404 Not Found

HTTP 404 is returned when the specified collection does not exist.

HTTP 404 is returned when the specified collection does not exist.

```
{  
  "error": "Unable to find the object that you requested."  
}
```

collections users-remove

DELETE /v1/collections/{collection_id}/users/{id}

Remove a user from a collection.

FLAGS

`--collection_id` [required] The numeric ID of the user collection, e.g. `--collection_id 7788`
`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 42618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py collections users-remove --collection_id 7788 --id 42618
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.
HTTP 204 confirms the operation completed successfully.

collections users-show

GET /v1/collections/{collection_id}/users/{id}

Show a collection user.

FLAGS

`--collection_id` [required] The numeric ID of the user collection, e.g. `--collection_id 7788`
`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 42618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py collections users-show --collection_id 7788 --id 42618
```

RESPONSE

HTTP 200 OK

Empty response — this is the expected behaviour when no users exist in the collection.
`[]`

current-user show

GET /v1/current_user

Retrieve a single current-user resource by its ID.

EXAMPLE COMMAND

```
python dnscli.py current-user show
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "42618",
    "type": "users",
    "uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
    "attributes": {
      "name": "Jane Smith",
      "email": "jane.smith@example.com",
      "email_verified": true,
      "first_name": "Jane",
      "last_name": "Smith",
      "phone": "+14035128790",
      "mfa_enabled": true,
      "created_at": "2020-11-23T19:40:02.381-05:00",
      "created_at_epoch_utc": 1606178402,
      "updated_at": "2026-05-21T17:21:26.365-04:00",
      "intercom_user_verification":
        "b480ce6135d61bb46b4fe92465d7dec45295c9880f07fff6759ad4ca575edeee",
      "last_sign_in_at": "2026-05-21T17:21:26.365-04:00",
      "metadata": {},
      "must_reset_password": false
    }
  }
}
```


current-user update

PATCH /v1/current_user

Update an existing current-user resource by its ID.

FLAGS

--first_name The user's given (first) name, e.g. --first_name "Jane"
--last_name The user's family (last) name, e.g. --last_name "Smith"
--phone A phone number. Example: +12025551234, e.g. --phone +12025551234

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py current-user update --first_name "Jane" --last_name "Smith"
```

RESPONSE

HTTP 200 OK

```
{
  "id": 42618,
  "type": "users",
  "attributes": {
    "first_name": "Jane",
    "last_name": "Smith",
    "updated_at": "2025-06-01T10:00:00.000-04:00"
  }
}
```

```
}  
}
```

dictionary qp-methods

GET /v1/dictionary/qp_methods

List QP method types.

EXAMPLE COMMAND

```
python dnscli.py dictionary qp-methods
```

RESPONSE

HTTP 200 OK

```
[
  {
    "id": 0,
    "allow": null,
    "name": "Reserved",
    "description": "Reserved"
  },
  {
    "id": 1,
    "allow": true,
    "name": "Whitelisted",
    "description": "Allowed - Whitelisted"
  },
  {
    "...": "(49 more)"
  }
]
```

domains bulk-lookup

GET /v1/domains/bulk_lookup

Classify multiple FQDNs in a single request..

FLAGS

`--fqdns [required]` Comma-separated list of FQDNs to classify, e.g. `--fqdns "example"`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py domains bulk-lookup
```

RESPONSE

HTTP 200 OK

```
{
  "data": []
}
```

domains suggest-threat

POST /v1/domains/suggest_threat

Suggest a domain for threat-intel review..

FLAGS

--fqdn [required] Fully-qualified domain name to flag, e.g. --fqdn "example"
--notes [required] Reason or notes for the threat suggestion, e.g. --notes "example"
--categories Comma-separated category IDs (optional), e.g. --categories "example"

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py domains suggest-threat
```

RESPONSE

HTTP 200 OK

```
{  
  "status": "ok",  
  "message": "Domain submitted for review"  
}
```

domains user-lookup

GET /v1/domains/user_lookup

Gets all domains associated with a particular FQDN..

FLAGS

--fqdn Fully-qualified domain name to look up, e.g. --fqdn dnsfilter.com

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py domains user-lookup --fqdn dnsfilter.com
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "",
    "type": "domains",
    "attributes": {
      "name": "dnsfilter.com",
      "host": "dnsfilter.com"
    },
    "relationships": {
```

```
"categories": {  
  "data": [  
    {  
      "id": "19",  
      "type": "categories"  
    }  
  ]  
},  
"applications": {  
  "data": []  
}  
}  
}
```

enterprise-connections create

POST /v1/enterprise_connections

Create a new enterprise-connections resource.

FLAGS

`--client_id` OAuth client ID, e.g. `--client_id my-client-id`
`--client_secret` OAuth client secret, e.g. `--client_secret my-secret`
`--discovery_url` OIDC discovery URL, e.g. `--discovery_url "https://idp.company.com/.well-known/openid-configuration"`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--default_organization_id` Default organization ID, e.g. `--default_organization_id 1`
`--strategy` Connection strategy (e.g. `oidc`, `saml`), e.g. `--strategy oidc`
`--display_name` Display name, e.g. `--display_name "Company SSO"`
`--role_default` Default role for new users, e.g. `--role_default read_only`
`--role_map` JSON array of role mapping rules, e.g. `--role_map ["admin-group:administrator"]`
`--idp` Identity provider identifier, e.g. `--idp okta`
`--authorized_domains` Authorized email domains, e.g. `--authorized_domains ["company.com"]`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND


```
python dnscli.py enterprise-connections create \  
--client_id my-client-id \  
--client_secret my-secret \  
--discovery_url "https://idp.company.com/.well-known/openid-configuration" \  
--strategy oidc \  
--display_name "Company SSO"
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 4401,  
  "type": "enterprise_connections",  
  "attributes": {  
    "display_name": "Company SSO",  
    "strategy": "oidc",  
    "status": "active"  
  }  
}
```

enterprise-connections delete

DELETE /v1/enterprise_connections/{id}

Permanently delete a enterprise-connections resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 4401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py enterprise-connections delete --id 4401
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

enterprise-connections list

GET /v1/enterprise_connections

Retrieve a paginated list of enterprise-connections resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25
--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py enterprise-connections list \  
  --page 1 --per_page 25 \  
  --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{  
  "data": []  
}
```

enterprise-connections list-all

GET /v1/enterprise_connections/all

Retrieve all enterprise-connections resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py enterprise-connections list-all
```

RESPONSE

HTTP 200 OK

Empty response — this is the expected behaviour when no SSO connections are configured.

```
{  
  "data": []  
}
```

enterprise-connections show

GET /v1/enterprise_connections/{id}

Retrieve a single enterprise-connections resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 4401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py enterprise-connections show --id 4401
```

RESPONSE

HTTP 200 OK

Empty response — this is the expected behaviour when no SSO connections are configured.

```
{
  "data": []
}
```

enterprise-connections update

PATCH /v1/enterprise_connections/{id}

Update an existing enterprise-connections resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 4401
--organization_id The numeric ID of the organization that owns this resource, e.g. --organization_id 802315
--default_organization_id Default organization ID, e.g. --default_organization_id 1
--display_name Connection display name, e.g. --display_name "Company SSO Updated"
--role_default Default role for new users, e.g. --role_default read_only
--role_map JSON array of role mapping rules, e.g. --role_map ["admin-group:administrator"]
--idp Identity provider identifier, e.g. --idp okta
--authorized_domains Authorized email domains, e.g. --authorized_domains ["company.com"]

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py enterprise-connections update \  
  --id 4401 \  
  --display_name "Company SSO Updated"
```

RESPONSE

HTTP 200 OK

```
{
  "id": 4401,
  "type": "enterprise_connections",
  "attributes": {
    "display_name": "Company SSO Updated"
  }
}
```

invoices current

GET /v1/invoices/current

Show current invoice.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py invoices current --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "month_to_date": {
    "id": null,
    "start_date": "2022-07-30",
    "end_date": "2022-08-29",
    "organization_id": 802315,
    "created_at": null,
    "updated_at": null,
```



```
"total_cents": null,
"stripe_id": null,
"requests": 0,
"paid": null,
"closed": null,
"forgiven": null,
"payment_attempts": null,
"payment_first_attempt": null,
"payment_last_attempt": null,
"payment_date": null,
"payment_receipt_number": null,
"organization_name": "Acme Accounting Co.",
"chargebee_draft_id": null,
"chargebee_invoice_id": null,
"price_above_base_plan_cents": null,
"base_plan_price_cents": null,
"chargebee_plan_id": null,
"has_usage": null,
"status": "unknown",
"write_off": false,
"write_off_amount_cents": null,
"write_off_at": null,
"write_off_adjustment_credit_notes": null,
"write_off_paid_by_invoice_id": null,
"write_off_paid_amount_cents": null,
"is_reactivation": false,
"reactivation_pending_invoices": null,
"paid_by_invoice_id": null,
"uuid": null,
"invoice_item_groups": [
  {
    "id": null,
    "invoice_id": null,
    "organization_name": "Acme Accounting Co.",
    "organization_id": 802315,
    "subtotal_cents": null,
    "created_at": null,
    "updated_at": null,
    "requests": 0,
    "price_above_base_plan_cents": null,
    "uuid": null,
```

```
"invoice_items": [  
  {  
    "id": null,  
    "invoice_id": null,  
    "network_name": "Acme Accounting Co. HQ",  
    "network_id": 736401,  
    "cents": null,  
    "requests": 0,  
    "created_at": null,  
    "updated_at": null,  
    "stripe_id": null,  
    ...  
  }  
]
```

invoices list

GET /v1/invoices

Retrieve a paginated list of invoices resources.

FLAGS

`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`
`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py invoices list --page 1 --per_page 25 --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{  
  "data": []  
}
```

invoices show

GET /v1/invoices/{id}

Retrieve a single invoices resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id inv-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py invoices show --id inv-001
```

RESPONSE

HTTP 404 Not Found

HTTP 404 is expected when the account has no invoices or billing is not configured.

HTTP 404 is expected when the account has no invoices or billing is not configured.

```
{
  "error": "Unable to find the object that you requested."
}
```

ip-addresses create

POST /v1/ip_addresses

Create a new ip-addresses resource.

FLAGS

`--address` [required] The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. `--address 203.0.113.5`
`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` [required] The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--dynamic_hostname` A dynamic DNS hostname associated with this IP address, e.g. `--dynamic_hostname myhost.dyndns.org`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`
`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`
`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses create \  
  --address 203.0.113.5 \  
  --organization_id 802315 \  
  --network_id 736401
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 88801,  
  "type": "ip_addresses",  
  "attributes": {  
    "address": "203.0.113.5",  
    "network_id": 736401,  
    "organization_id": 802315  
  }  
}
```

ip-addresses delete

DELETE /v1/ip_addresses/{id}

Permanently delete a ip-addresses resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 2100638

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses delete --id 2100638
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

ip-addresses list

GET /v1/ip_addresses

Retrieve a paginated list of ip-addresses resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "2100638",
      "type": "ip_addresses",
      "uuid": "0198e2e7-4984-7aad-b8e7-2f60b60a7960",
      "attributes": {
        "ip": "174.0.40.254",
```



```

    "address": "174.0.40.254",
    "network_id": 9710205,
    "organization_id": 802315
  },
  "relationships": {
    "network": {
      "data": {
        "id": "9710205",
        "type": "networks",
        "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"
      }
    }
  }
},
{
  "id": "2163176",
  "type": "ip_addresses",
  "uuid": "019a311b-d5ae-741f-a107-235a757fdcbd",
  "attributes": {
    "ip": "98.124.127.0",
    "address": "98.124.127.0",
    "network_id": 9710205,
    "organization_id": 802315
  },
  "relationships": {
    "network": {
      "data": {
        "id": "9710205",
        "type": "networks",
        "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"
      }
    }
  }
},
{
  "id": "2163177",
  "type": "ip_addresses",
  "uuid": "019a311b-d5bd-7b1b-ae11-6648c4eeac7d",
  "attributes": {
    "ip": "98.124.127.1",
    "address": "98.124.127.1",

```

```
"network_id": 9710205,  
"organization_id": 802315  
,  
"relationships": {  
  "network": {  
    "data": {  
      "id": "9710205",  
      "type": "networks",  
      "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"  
    }  
  }  
  ...  
}
```

ip-addresses list-all

GET /v1/ip_addresses/all

Retrieve all ip-addresses resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py ip-addresses list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "2100638",
      "type": "ip_addresses",
      "uuid": "0198e2e7-4984-7aad-b8e7-2f60b60a7960",
      "attributes": {
        "ip": "174.0.40.254",
        "address": "174.0.40.254",
        "network_id": 9710205,
        "organization_id": 802315
      },
      "relationships": {
        "network": {
          "data": {
            "id": "9710205",
            "type": "networks",
            "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"
          }
        }
      }
    },
    {
      "id": "2163176",
      "type": "ip_addresses",
```

```

"uuid": "019a311b-d5ae-741f-a107-235a757fdcbd",
"attributes": {
  "ip": "98.124.127.0",
  "address": "98.124.127.0",
  "network_id": 9710205,
  "organization_id": 802315
},
"relationships": {
  "network": {
    "data": {
      "id": "9710205",
      "type": "networks",
      "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"
    }
  }
}
},
{
  "id": "2163177",
  "type": "ip_addresses",
  "uuid": "019a311b-d5bd-7b1b-ae11-6648c4eeac7d",
  "attributes": {
    "ip": "98.124.127.1",
    "address": "98.124.127.1",
    "network_id": 9710205,
    "organization_id": 802315
  },
  "relationships": {
    "network": {
      "data": {
        "id": "9710205",
        "type": "networks",
        "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"
      }
    }
  }
}
...

```

ip-addresses myip

GET /v1/ip_addresses/myip

Show caller's IP address.

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses myip
```

RESPONSE

HTTP 200 OK

```
{  
  "myip": "174.0.40.117"  
}
```

ip-addresses show

GET /v1/ip_addresses/{id}

Retrieve a single ip-addresses resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 2100638

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses show --id 2100638
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "2100638",
    "type": "ip_addresses",
    "uuid": "0198e2e7-4984-7aad-b8e7-2f60b60a7960",
    "attributes": {
      "ip": "174.0.40.254",
      "address": "174.0.40.254",
      "network_id": 9710205,
```

```
"organization_id": 802315
},
"relationships": {
  "network": {
    "data": {
      "id": "9710205",
      "type": "networks",
      "uuid": "0198e2e7-497b-73b9-b6f8-f6386dc7098d"
    }
  }
}
}
```

ip-addresses update

PATCH /v1/ip_addresses/{id}

Update an existing ip-addresses resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 2100638
--address [required] The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. --address 203.0.113.5
--organization_id The numeric ID of the organization that owns this resource, e.g. --organization_id 802315
--network_id The numeric ID of the network to associate with, e.g. --network_id 736401
--dynamic_hostname A dynamic DNS hostname associated with this IP address, e.g. --dynamic_hostname myhost.dyndns.org

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses update --id 2100638 --address 203.0.113.5
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 88801,
```



```
"type": "ip_addresses",  
"attributes": {  
  "address": "203.0.113.5",  
  "dynamic_hostname": "updated.example.com"  
}  
}
```

ip-addresses verify

GET /v1/ip_addresses/verify

Verify whether an IP address is registered..

FLAGS

`--address [required]` The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. `--address 203.0.113.5`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py ip-addresses verify
```

RESPONSE

HTTP 500 Internal Server Error

```
{
  "status": 500,
  "error": "Internal Server Error"
}
```

mac-addresses create

POST /v1/mac_addresses

Create a new mac-addresses resource.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--address` The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. `--address AA:BB:CC:DD:EE:FF`
`--filter_value` A display label for this MAC address, shown in the dashboard, e.g. `--filter_value "Reception Printer"`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py mac-addresses create \  
  --organization_id 802315 \  
  --address AA:BB:CC:DD:EE:FF \  
  --filter_value "Reception Printer" \  
  --policy_id 285109
```

RESPONSE

HTTP 201 Created

```
{
  "id": 77701,
  "type": "mac_addresses",
  "attributes": {
    "address": "AA:BB:CC:DD:EE:FF",
    "filter_value": "Reception Printer",
    "policy_id": 285109,
    "organization_id": 802315
  }
}
```

mac-addresses delete

DELETE /v1/mac_addresses/{id}

Permanently delete a mac-addresses resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 77701`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py mac-addresses delete --id 77701
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

mac-addresses list

GET /v1/mac_addresses

Retrieve a paginated list of mac-addresses resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py mac-addresses list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/mac_addresses?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",
    "first": "https://api.dnsfilter.com/v1/mac_addresses?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",
    "prev": null,
```

```
"next": null,  
"last": "https://api.dnsfilter.com/v1/mac_addresses?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2"  
}  
}
```

mac-addresses list-all

GET /v1/mac_addresses/all

Retrieve all mac-addresses resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py mac-addresses list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/mac_addresses/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "first": "https://api.dnsfilter.com/v1/mac_addresses/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "prev": null,
    "next": null,
    "last": "https://api.dnsfilter.com/v1/mac_addresses/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500"
  }
}
```


mac-addresses show

GET /v1/mac_addresses/{id}

Retrieve a single mac-addresses resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 77701

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py mac-addresses show --id 77701
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/mac_addresses?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=1",
    "first": "https://api.dnsfilter.com/v1/mac_addresses?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=1",
    "prev": null,
    "next": null,
```

```
"last": "https://api.dnsfilter.com/v1/mac_addresses?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=1"  
}  
}
```

mac-addresses update

PATCH /v1/mac_addresses/{id}

Update an existing mac-addresses resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 77701`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--address` The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. `--address AA:BB:CC:DD:EE:FF`
`--filter_value` A display label for this MAC address, shown in the dashboard, e.g. `--filter_value "Updated Label"`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py mac-addresses update \  
  --id 77701 \  
  --address AA:BB:CC:DD:EE:FF \  
  --filter_value "Updated Label"
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 77701,  
  "type": "mac_addresses",  
  "attributes": {  
    "address": "AA:BB:CC:DD:EE:FF",  
    "filter_value": "Updated Label"  
  }  
}
```

metrics org-usage

GET /v1/metrics/organization_usage/{id}

Organization usage metrics.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 802315
--from [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. --from 10.0.1.0
--to [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. --to 10.0.1.255

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py metrics org-usage --id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_id": 802315,
    "plan": "undeclared",
    "term": "monthly",
```

```
"user_count": 1,  
"wifi_count": 0,  
"total_requests": 0,  
"estimated_users": 0,  
"from": "2025-01-01",  
"to": "2025-01-31",  
"days_in_period": 31  
},  
"meta": {  
  "queries_per_user": 6000  
}  
}
```

metrics org-usage-detailed

GET /v1/metrics/organization_usage_detailed/{id}

Detailed organization usage metrics.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 802315
--from [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. --from 10.0.1.0
--to [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. --to 10.0.1.255

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py metrics org-usage-detailed --id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_id": 802315,
    "plan": "free-filtering",
    "term": "not_applicable",
```

```
"allocated_network_traffic": 0,  
"network_traffic_in_use": 13,  
"allocated_roaming_clients": 1,  
"roaming_clients_in_use": 12,  
"user_count": 1,  
"wifi_count": 0,  
"from": "2025-01-01",  
"to": "2025-01-31",  
"days_in_period": 31  
},  
"meta": {  
  "queries_per_user": 10000  
}  
}
```


networks bulk-create

POST /v1/networks/bulk_create

Bulk create networks.

EXAMPLE COMMAND

```
python dnscli.py networks bulk-create
```

RESPONSE

HTTP 202 Accepted

```
{
  "id": "job-bc-123",
  "status": "pending",
  "created_at": "2025-06-01T10:00:00.000-04:00"
}
```

networks bulk-create-show

GET /v1/networks/bulk_create/{id}

Show a bulk-create job.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id job-bc-123`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks bulk-create-show --id job-bc-123
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-bc-123",
  "status": "completed",
  "created_count": 3,
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:08.000-04:00"
}
```

networks bulk-destroy

DELETE /v1/networks/bulk_destroy

Bulk delete networks..

FLAGS

--ids [required] An array of resource IDs. Example: ["id1","id2"], e.g. **--ids ["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]**
--organization_id The numeric ID of the organization that owns this resource, e.g. **--organization_id 802315**

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. **--raw**
--json Output JSON to stdout (automatic when piped), e.g. **--json**
--to-csv FILE Write the response to a CSV file, e.g. **--to-csv output.csv**
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. **--from-csv policies.csv**
--template Print a blank CSV import template for this command and exit, e.g. **--template**
--filter EXPR Keep matching rows only, e.g. **--filter status=active**
--columns a,b,c Limit table output to named columns, e.g. **--columns id,name**
--org-id ID Override the stored organization ID for this call only, e.g. **--org-id 802315**

EXAMPLE COMMAND

```
python dnsfcli.py networks bulk-destroy
```

RESPONSE

HTTP 202 Accepted

```
{
  "id": "job-bd-789",
  "status": "pending",
  "created_at": "2025-06-01T10:00:00.000-04:00"
}
```

networks bulk-destroy-show

GET /v1/networks/bulk_destroy/{id}

Show a bulk-destroy job.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id job-bd-789`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks bulk-destroy-show --id job-bd-789
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-bd-789",
  "status": "completed",
  "deleted_count": 5,
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:07.000-04:00"
}
```

networks bulk-update

POST /v1/networks/bulk_update

Bulk update networks.

FLAGS

`--ids` [required] An array of resource IDs. Example: ["id1","id2"], e.g. `--ids 736401,736402`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`
`--is_legacy_vpn_active` Enable legacy VPN mode for this network, e.g. `--is_legacy_vpn_active false`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks bulk-update --ids 736401,736402 --policy_id 285109
```

RESPONSE

HTTP 202 Accepted

```
{  
  "id": "job-bu-456",  
  "status": "pending",  
  "created_at": "2025-06-01T10:00:00.000-04:00"  
}
```

networks bulk-update-show

GET /v1/networks/bulk_update/{id}

Show a bulk-update job.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id job-bu-456`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks bulk-update-show --id job-bu-456
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-bu-456",
  "status": "completed",
  "updated_count": 5,
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:06.000-04:00"
}
```

networks counts

GET /v1/networks/counts

Network counts.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks counts
```

RESPONSE

HTTP 200 OK

```
{
  "all": 2,
  "protected": {
    "network_ids": [
      9710205
    ],
    "count": 1
  },
}
```



```
"unprotected": {  
  "network_ids": [],  
  "count": 0  
},  
"offline": 1  
}
```

networks create

POST /v1/networks

Create a new networks resource.

FLAGS

--name [required] A human-readable display name for this resource, e.g. --name "HQ Network"

--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315

--block_page_id The numeric ID of the block page to display when a domain is blocked, e.g. --block_page_id 5147

--policy_ids An array of policy IDs to assign. Example: ["285109","331207"], e.g. --policy_ids '["285109"]'

--external_id An external identifier from a third-party system, e.g. --external_id ext-12345

--is_legacy_vpn_active Enable legacy VPN mode for this network, e.g. --is_legacy_vpn_active false

--physical_address The physical street address of the network location, e.g. --physical_address "123 Main St, Denver CO"

--ip_addresses_attributes JSON array of IP address objects, e.g. --ip_addresses_attributes ["item1","item2"]

--local_domains Local domain names that should resolve internally, not via DNSFilter, e.g. --local_domains ["corp.local","internal.local"]

--local_resolvers IP addresses of local DNS resolvers to use for local_domains, e.g. --local_resolvers ["192.168.1.1","192.168.1.2"]

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnscli.py networks create \  
  --name "HQ Network" \  
  --organization_id 802315 \  
  --policy_ids '["285109"]' \  
  --physical_address "123 Main St, Denver CO"
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 9999901,  
  "type": "networks",  
  "attributes": {  
    "name": "HQ Network",  
    "organization_id": 802315,  
    "created_at": "2025-06-01T10:00:00.000-04:00"  
  }  
}
```

networks delete

DELETE /v1/networks/{id}

Permanently delete a networks resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 736401`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks delete --id 736401
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

networks geo

GET /v1/networks/geo

Network geo information.

EXAMPLE COMMAND

```
python dnscli.py networks geo
```

RESPONSE

HTTP 200 OK

```
{  
  "data": []  
}
```

networks lan-ip-show

GET /v1/networks/{id}/lan_ips/{lan_ip_id}

Show a LAN IP.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401
--lan_ip_id [required] The numeric ID of the LAN IP address entry, e.g. --lan_ip_id 9901

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks lan-ip-show --id 736401 --lan_ip_id 9901
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/networks/736401/lan_ips?
page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "first": "https://api.dnsfilter.com/v1/networks/736401/lan_ips?
page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "prev": null,
```

```
"next": null,  
"last": "https://api.dnsfilter.com/v1/networks/736401/lan_ips?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500"  
}  
}
```

networks lan-ip-update

PATCH /v1/networks/{id}/lan_ips/{lan_ip_id}

Update a LAN IP.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401
--lan_ip_id [required] The numeric ID of the LAN IP address entry, e.g. --lan_ip_id 9901
--name A human-readable display name for this resource, e.g. --name "Reception Desk"

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks lan-ip-update \  
  --id 736401 \  
  --lan_ip_id 9901 \  
  --name "Reception Desk"
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 9901,  
  "type": "lan_ips",  
  "attributes": {
```



```
"name": "Reception Desk",  
"updated_at": "2025-06-01T10:00:00.000-04:00"  
}  
}
```

networks lan-ips

GET /v1/networks/{id}/lan_ips

List LAN IPs for a network.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks lan-ips --id 736401
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/networks/736401/lan_ips?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "first": "https://api.dnsfilter.com/v1/networks/736401/lan_ips?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "prev": null,
    "next": null,
  }
}
```

```
"last": "https://api.dnsfilter.com/v1/networks/736401/lan_ips?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500"  
}  
}
```

networks list

GET /v1/networks

Retrieve a paginated list of networks resources.

FLAGS

`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "736401",
      "type": "networks",
      "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e",
```

```
"attributes": {
  "name": "Acme Accounting Co. HQ",
  "physical_address": "",
  "latitude": null,
  "longitude": null,
  "is_legacy_vpn_active": false,
  "block_default_appearance": true,
  "block_org_name": null,
  "block_email_addr": null,
  "block_logo_uuid": null,
  "block_page_id": null,
  "allow_all_policies": false,
  "scheduled_policy_id": null,
  "policy_id": 1486063,
  "secret_key": "15d6b45d316a730ae890b14e",
  "deleted_at": null,
  "billing_per_user": null,
  "billing_user_count": null,
  "network_type_units": null,
  "network_type": "default",
  "pin": "908662",
  "sitekey_auto_register_expires_at": null,
  "local_domains": [
    "beef.sol-local",
    "sandwich.deli.yummyum"
  ],
  "local_resolvers": [
    "9.9.9.9"
  ],
  "external_id": "",
  "description": null,
  "truncated_ip_count": 0,
  "ip_count": 0
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  }
}
```

```
},
"scheduled_policy": {
  "data": null
},
"ip_addresses": {
  "data": [
    {
      "id": "2163165",
      "type": "ip_addresses",
      "uuid": "019a3102-4a42-73e8-96a0-4d34124d750b"
    }
  ]
},
...
```

networks list-all

GET /v1/networks/all

Retrieve all networks resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py networks list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "510520",
      "type": "networks",
      "uuid": "0175f7b3-31e5-7c07-a3c1-72a4457306a0",
      "attributes": {
        "name": "Acme Accounting Co. HQ",
        "physical_address": "",
        "latitude": null,
        "longitude": null,
        "is_legacy_vpn_active": false,
        "block_default_appearance": true,
        "block_org_name": null,
        "block_email_addr": null,
        "block_logo_uuid": null,
        "block_page_id": null,
        "allow_all_policies": false,
        "scheduled_policy_id": null,
        "policy_id": null,
        "secret_key": null,
        "deleted_at": "2020-11-23T19:43:54.916-05:00",
        "billing_per_user": null,
        "billing_user_count": null,
        "network_type_units": null,
      }
    }
  ]
}
```

```
"network_type": "default",
"pin": "550123",
"sitekey_auto_register_expires_at": null,
"local_domains": [],
"local_resolvers": [],
"external_id": null,
"description": null,
"truncated_ip_count": 0,
"ip_count": 0
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  },
  "scheduled_policy": {
    "data": null
  },
  "ip_addresses": {
    "data": []
  },
  "policy": {
    "data": null
  },
  "block_page": {
    "data": null
  },
  "policies": {
    "data": []
  }
}
},
...
```


networks lookup

GET /v1/networks/lookup

Lookup a network.

FLAGS

`--requesting_ip_address` [required] IP address to look up, e.g. `--requesting_ip_address "example"`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks lookup
```

RESPONSE

HTTP 400 Bad Request

HTTP 400 is returned when the IP address is not registered to any network on this account.

HTTP 400 is returned when the IP address is not registered to any network on this account.

networks msp

GET /v1/networks/msp

List MSP networks.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks msp --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "736401",
      "type": "networks",
      "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e",
      "attributes": {
        "name": "Acme Accounting Co. HQ",
```

```
"physical_address": "",
"latitude": null,
"longitude": null,
"is_legacy_vpn_active": false,
"block_default_appearance": true,
"block_org_name": null,
"block_email_addr": null,
"block_logo_uuid": null,
"block_page_id": null,
"allow_all_policies": false,
"scheduled_policy_id": null,
"policy_id": 1486063,
"secret_key": "15d6b45d316a730ae890b14e",
"deleted_at": null,
"billing_per_user": false,
"billing_user_count": 0,
"network_type_units": 0,
"network_type": "default",
"pin": "908662",
"sitekey_auto_register_expires_at": null,
"local_domains": [
  "beef.sol-local",
  "sandwich.deli.yummyum"
],
"local_resolvers": [
  "9.9.9.9"
],
"external_id": "",
"description": null,
"truncated_ip_count": 0,
"ip_count": 0
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  }
},
"scheduled_policy": {
```

```
"data": null
},
"ip_addresses": {
  "data": [
    {
      "id": "2163165",
      "type": "ip_addresses",
      "uuid": "019a3102-4a42-73e8-96a0-4d34124d750b"
    }
  ]
},
...
```

networks msp-all

GET /v1/networks/msp/all

List all MSP networks.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks msp-all --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "510520",
      "type": "networks",
      "uuid": "0175f7b3-31e5-7c07-a3c1-72a4457306a0",
      "attributes": {
        "name": "Acme Accounting Co. HQ",
```

```

    "physical_address": "",
    "latitude": null,
    "longitude": null,
    "is_legacy_vpn_active": false,
    "block_default_appearance": true,
    "block_org_name": null,
    "block_email_addr": null,
    "block_logo_uuid": null,
    "block_page_id": null,
    "allow_all_policies": false,
    "scheduled_policy_id": null,
    "policy_id": null,
    "secret_key": null,
    "deleted_at": "2020-11-23T19:43:54.916-05:00",
    "billing_per_user": false,
    "billing_user_count": 0,
    "network_type_units": 0,
    "network_type": "default",
    "pin": "550123",
    "sitekey_auto_register_expires_at": null,
    "local_domains": [],
    "local_resolvers": [],
    "external_id": null,
    "description": null,
    "truncated_ip_count": 0,
    "ip_count": 0
  },
  "relationships": {
    "organization": {
      "data": {
        "id": "802315",
        "type": "organizations",
        "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
      }
    }
  },
  "scheduled_policy": {
    "data": null
  },
  "ip_addresses": {
    "data": []
  },

```

```
"policy": {  
  "data": null  
},  
"block_page": {  
  "data": null  
},  
"policies": {  
  "data": []  
}  
}  
},  
...
```

networks secret-key-create

POST /v1/networks/{id}/secret_key

Create network secret key.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks secret-key-create --id 736401
```

RESPONSE

HTTP 201 Created

```
{
  "secret_key": "sk-a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6"
}
```


networks secret-key-delete

DELETE /v1/networks/{id}/secret_key

Delete network secret key.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks secret-key-delete --id 736401
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

networks secret-key-update

PATCH /v1/networks/{id}/secret_key

Update network secret key.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks secret-key-update --id 736401
```

RESPONSE

HTTP 200 OK

```
{
  "secret_key": "sk-newkey1234567890abcdef1234567890"
}
```

networks show

GET /v1/networks/{id}

Retrieve a single networks resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks show --id 736401
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "736401",
    "type": "networks",
    "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e",
    "attributes": {
      "name": "Acme Accounting Co. HQ",
      "physical_address": "",
      "latitude": null,
```

```
"longitude": null,
"is_legacy_vpn_active": false,
"block_default_appearance": true,
"block_org_name": null,
"block_email_addr": null,
"block_logo_uuid": null,
"block_page_id": null,
"allow_all_policies": false,
"scheduled_policy_id": null,
"policy_id": 1486063,
"secret_key": "15d6b45d316a730ae890b14e",
"deleted_at": null,
"billing_per_user": null,
"billing_user_count": null,
"network_type_units": null,
"network_type": "default",
"pin": "908662",
"sitekey_auto_register_expires_at": null,
"local_domains": [
  "beef.sol-local",
  "sandwich.deli.yummyum"
],
"local_resolvers": [
  "9.9.9.9"
],
"external_id": "",
"description": null,
"truncated_ip_count": 0,
"ip_count": 0
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  }
},
"scheduled_policy": {
  "data": null
},
```

```
"ip_addresses": {  
  "data": [  
    {  
      "id": "2163165",  
      "type": "ip_addresses",  
      "uuid": "019a3102-4a42-73e8-96a0-4d34124d750b"  
    }  
  ]  
},  
"policy": {  
  ...  
}
```

networks subnets

GET /v1/networks/subnets

List all subnets.

EXAMPLE COMMAND

```
python dnscli.py networks subnets
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "2165762",
      "type": "network_subnets",
      "uuid": "01997ca0-8b4a-78b2-91fa-b07e84cba994",
      "attributes": {
        "name": "Smith",
        "version": 4,
        "from": "192.168.1.12",
        "to": "192.168.1.14",
        "created_at": "2025-09-24T12:48:40.266-04:00",
        "updated_at": "2025-09-24T12:48:40.266-04:00"
      },
      "relationships": {
        "network": {
          "data": {
            "id": "736401",
            "type": "networks",
            "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e"
          }
        }
      },
      "policy": {
        "data": null
      }
    }
  ],
}
```

```
"scheduled_policy": {  
  "data": null  
},  
"block_page": {  
  "data": null  
}  
}  
]  
}
```

networks subnets-create

POST /v1/networks/{id}/subnets

Add a subnet to a network.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 736401`
`--name` [required] A human-readable display name for this resource, e.g. `--name "Sales Floor"`
`--from` [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. `--from 10.0.1.0`
`--to` [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. `--to 10.0.1.255`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks subnets-create \  
  --id 736401 \  
  --name "Sales Floor" \  
  --from 10.0.1.0 \  
  --to 10.0.1.255
```



```
--from 10.0.1.0 \  
--to 10.0.1.255
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 7701,  
  "type": "network_subnets",  
  "attributes": {  
    "name": "Sales Floor",  
    "from": "10.0.1.0",  
    "to": "10.0.1.255",  
    "policy_id": 285109  
  }  
}
```

networks subnets-delete

DELETE /v1/networks/{id}/subnets/{subnet_id}

Delete a subnet.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401
--subnet_id [required] The numeric ID of the network subnet, e.g. --subnet_id 2165762

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks subnets-delete --id 736401 --subnet_id 2165762
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.
HTTP 204 confirms the operation completed successfully.

networks subnets-list

GET /v1/networks/{id}/subnets

List subnets for a network.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks subnets-list --id 736401
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "2165762",
      "type": "network_subnets",
      "uuid": "01997ca0-8b4a-78b2-91fa-b07e84cba994",
      "attributes": {
        "name": "Smith",
        "version": 4,
```

```

    "from": "192.168.1.12",
    "to": "192.168.1.14",
    "created_at": "2025-09-24T12:48:40.266-04:00",
    "updated_at": "2025-09-24T12:48:40.266-04:00"
  },
  "relationships": {
    "network": {
      "data": {
        "id": "736401",
        "type": "networks",
        "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e"
      }
    },
    "policy": {
      "data": null
    },
    "scheduled_policy": {
      "data": null
    },
    "block_page": {
      "data": null
    }
  }
},
"links": {
  "self": "https://api.dnsfilter.com/v1/networks/736401/subnets?
page%5Bnumber%5D=1&page%5Bsize%5D=1500",
  "first": "https://api.dnsfilter.com/v1/networks/736401/subnets?
page%5Bnumber%5D=1&page%5Bsize%5D=1500",
  "prev": null,
  "next": null,
  "last": "https://api.dnsfilter.com/v1/networks/736401/subnets?
page%5Bnumber%5D=1&page%5Bsize%5D=1500"
}
}

```

networks subnets-show

GET /v1/networks/{id}/subnets/{subnet_id}

Show a subnet.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 736401
--subnet_id [required] The numeric ID of the network subnet, e.g. --subnet_id 2165762

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks subnets-show --id 736401 --subnet_id 2165762
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "2165762",
    "type": "network_subnets",
    "uuid": "01997ca0-8b4a-78b2-91fa-b07e84cba994",
    "attributes": {
      "name": "Smith",
      "version": 4,
```

```
"from": "192.168.1.12",
"to": "192.168.1.14",
"created_at": "2025-09-24T12:48:40.266-04:00",
"updated_at": "2025-09-24T12:48:40.266-04:00"
},
"relationships": {
  "network": {
    "data": {
      "id": "736401",
      "type": "networks",
      "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e"
    }
  },
  "policy": {
    "data": null
  },
  "scheduled_policy": {
    "data": null
  },
  "block_page": {
    "data": null
  }
}
}
```

networks subnets-update

PATCH /v1/networks/{id}/subnets/{subnet_id}

Update a subnet.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 736401`
`--subnet_id` [required] The numeric ID of the network subnet, e.g. `--subnet_id 2165762`
`--name` [required] A human-readable display name for this resource, e.g. `--name "Sales Floor Renamed"`
`--from` [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. `--from 10.0.1.0`
`--to` [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. `--to 10.0.1.255`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py networks subnets-update \  
  --id 736401 \  
  --subnet_id 2165762 \  
  --name "Sales Floor Renamed" \  
  --from 10.0.1.0 \  
  --to 10.0.1.255 \  
  --policy_id 285109 \  
  --scheduled_policy_id 71203 \  
  --block_page_id 5147
```

```
--name "Sales Floor Renamed" \  
--from 10.0.1.0 \  
--to 10.0.1.255
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 7701,  
  "type": "network_subnets",  
  "attributes": {  
    "name": "Sales Floor Renamed",  
    "from": "10.0.1.0",  
    "to": "10.0.1.255"  
  }  
}
```


networks update

PATCH /v1/networks/{id}

Update an existing networks resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 736401`
`--name` [required] A human-readable display name for this resource, e.g. `--name "HQ Network Updated"`
`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`
`--policy_ids` An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids ["285109","331207"]`
`--external_id` An external identifier from a third-party system, e.g. `--external_id ext-12345`
`--is_legacy_vpn_active` Enable legacy VPN mode for this network, e.g. `--is_legacy_vpn_active false`
`--physical_address` The physical street address of the network location, e.g. `--physical_address "123 Main St, Denver CO"`
`--ip_addresses_attributes` JSON array of IP address objects, e.g. `--ip_addresses_attributes ["item1","item2"]`
`--local_domains` Local domain names that should resolve internally, not via DNSFilter, e.g. `--local_domains ["corp.local","internal.local"]`
`--local_resolvers` IP addresses of local DNS resolvers to use for local_domains, e.g. `--local_resolvers ["192.168.1.1","192.168.1.2"]`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py networks update \  
  --id 736401 \  
  --name "HQ Network Updated" \  
  --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 736401,  
  "type": "networks",  
  "attributes": {  
    "name": "HQ Network Updated",  
    "updated_at": "2025-06-01T10:00:00.000-04:00"  
  }  
}
```

organizations bulk-update

PATCH /v1/organizations/bulk_update

Bulk update organizations.

FLAGS

--organization_ids [required] An array of organization IDs. Example: ["802315","802316"], e.g. --organization_ids '["802315"]'
--msp_id MSP ID, e.g. --msp_id 8801
--exclude_organization_ids Array of org IDs to exclude, e.g. --exclude_organization_ids ["item1","item2"]
--vpn_settings_state_type_id VPN state type ID, e.g. --vpn_settings_state_type_id 1
--gdpr Enable GDPR, e.g. --gdpr true
--send_uninstall_notifications_to_admin_users Send uninstall notifications, e.g. --send_uninstall_notifications_to_admin_users true
--show_pii_rc_hostnames Show PII hostnames, e.g. --show_pii_rc_hostnames false
--user_agent_uninstall_notification Uninstall notification, e.g. --user_agent_uninstall_notification true
--user_agent_uninstall_notification_recipient_emails Notification emails, e.g. --user_agent_uninstall_notification_recipient_emails ["item1","item2"]
--user_agents_auto_update Auto-update agents, e.g. --user_agents_auto_update true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations bulk-update --organization_ids '["802315"]' --gdpr true
```

RESPONSE

HTTP 200 OK

```
{  
  "status": "ok",  
  "updated_count": 2  
}
```

organizations cancel

POST /v1/organizations/{id}/cancel

Cancel an organization.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 802315

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations cancel --id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "id": 802315,
  "type": "organizations",
  "attributes": {
    "status": "cancelled"
  }
}
```

organizations create

POST /v1/organizations

Create a new organizations resource.

FLAGS

--name [required] A human-readable display name for this resource, e.g. --name "New Client Corp"

--billing_contact_name Billing contact name, e.g. --billing_contact_name "Jane Smith"

--billing_contact_phone Billing contact phone, e.g. --billing_contact_phone +12025551234

--billing_contact_email Billing contact email, e.g. --billing_contact_email admin@company.com

--address The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. --address 203.0.113.5

--managed_by_msp_id Managing MSP ID, e.g. --managed_by_msp_id 8801

--show_pii_rc_hostnames Show PII roaming client hostnames, e.g. --show_pii_rc_hostnames false

--unique_id External unique identifier, e.g. --unique_id ext-client-001

--sku A product SKU code. Example: professional, e.g. --sku professional

--quantity Seat quantity, e.g. --quantity 25

--gdpr Enable GDPR mode, e.g. --gdpr true

--privacy_mode Privacy mode (standard / strict), e.g. --privacy_mode standard

--enable_cybersight Enable CyberSight, e.g. --enable_cybersight true

--vpn_settings_organization_attributes VPN settings JSON object, e.g. --vpn_settings_organization_attributes {"enabled":false}

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnscli.py organizations create \  
  --name "New Client Corp" \  
  --billing_contact_email admin@company.com \  
  --sku professional
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 9999902,  
  "type": "organizations",  
  "attributes": {  
    "name": "New Client Corp",  
    "sku": "professional",  
    "created_at": "2025-06-01T10:00:00.000-04:00"  
  }  
}
```

organizations delete

DELETE /v1/organizations/{id}

Permanently delete a organizations resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 802315

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations delete --id 802315
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

organizations list

GET /v1/organizations

Retrieve a paginated list of organizations resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f",
      "attributes": {
        "name": "Acme Accounting Co.",

```

```
"stripe_customer_id": "cus_IRj1sm3XldF08O",
"feature_flags": [
  "user_pricing",
  "manual_subscriptions"
],
"msp_feature_flags": [],
"managed_by_msp_id": null,
"owned_msp_id": null,
"canceled": false,
"canceled_at": null,
"will_cancel_at": null,
"address": "",
"trial_days": null,
"billing_address": null,
"billing_manual": false,
"billing_contact_name": "",
"billing_contact_phone": "",
"billing_contact_email": "",
"first_traffic_sent": "2020-11-23T19:43:01.948-05:00",
"gdpr": false,
"summary_email_enabled": false,
"summary_email_address": null,
"release_channel": "stable",
"sync_tools_auto_update": true,
"user_agents_auto_update": true,
"usage_alert_cents": null,
"unbounce_data": null,
"plan_details": null,
"update_payment": true,
"industry": "5",
"external_id": null,
"metadata": null,
"unique_id": "4f53f068-6636-4a6e-b226-00f7732a9b03",
"sso_enabled": false,
"allowlist_updated_at": "2025-01-09T15:46:38.201-05:00",
"blocklist_updated_at": null,
"send_uninstall_notifications_to_admin_users": false,
"user_agent_uninstall_notification": true,
"user_agent_uninstall_notification_recipient_emails": [
  "jane.smith@example.com"
],
```

```
"pin_protected_uninstall": true,  
"fetch_onboard_details": true,  
"active_psa_integration": false,  
"show_pii_rc_hostnames": false,  
"segment": "regular-2024-phase-one",  
"privacy_mode": "standard",  
"vpn_settings_organization": null,  
"entitlements_enabled": true,  
"role": "owner",  
"current_mrr": 0,  
"current_license": "",  
...
```

organizations list-all

GET /v1/organizations/all

Retrieve all organizations resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py organizations list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f",
      "attributes": {
        "name": "Acme Accounting Co.",
        "stripe_customer_id": "cus_IRj1sm3XldF08O",
        "feature_flags": [
          "user_pricing",
          "manual_subscriptions"
        ],
        "msp_feature_flags": [],
        "managed_by_msp_id": null,
        "owned_msp_id": null,
        "canceled": false,
        "canceled_at": null,
        "will_cancel_at": null,
        "address": "",
        "trial_days": null,
        "billing_address": null,
        "billing_manual": false,
        "billing_contact_name": "",
        "billing_contact_phone": ""
      }
    }
  ]
}
```

```
"billing_contact_email": "",
"first_traffic_sent": "2020-11-23T19:43:01.948-05:00",
"gdpr": false,
"summary_email_enabled": false,
"summary_email_address": null,
"release_channel": "stable",
"sync_tools_auto_update": true,
"user_agents_auto_update": true,
"usage_alert_cents": null,
"unbounce_data": null,
"plan_details": null,
"update_payment": true,
"industry": "5",
"external_id": null,
"metadata": null,
"unique_id": "4f53f068-6636-4a6e-b226-00f7732a9b03",
"sso_enabled": false,
"allowlist_updated_at": "2025-01-09T15:46:38.201-05:00",
"blocklist_updated_at": null,
"send_uninstall_notifications_to_admin_users": false,
"user_agent_uninstall_notification": true,
"user_agent_uninstall_notification_recipient_emails": [
  "jane.smith@example.com"
],
"pin_protected_uninstall": true,
"fetched_onboard_details": true,
"active_psa_integration": false,
"show_pii_rc_hostnames": false,
"segment": "regular-2024-phase-one",
"privacy_mode": "standard",
"vpn_settings_organization": null,
"entitlements_enabled": true
},
"relationships": {
  "networks": {
    ...
```

organizations settings

GET /v1/organizations/settings

Show organization settings.

EXAMPLE COMMAND

```
python dnscli.py organizations settings
```

RESPONSE

HTTP 200 OK

```
[
  {
    "id": 802315,
    "name": "Acme Accounting Co.",
    "user_agents_auto_update": true,
    "user_agent_uninstall_notification": true,
    "send_uninstall_notifications_to_admin_users": false,
    "diagnostics_sharing_enabled": true,
    "remote_diagnostics_enabled": true,
    "pin_protected_uninstall": true,
    "connection_method": "loopback",
    "filtering_method": "dns_filtering",
    "fail_over_method": "fail_close",
    "enable_cybersight": null,
    "enable_browser_extension": null,
    "url_filtering": null,
    "ip_filtering": null,
    "privacy_mode": "standard",
    "diagnostics_level": "trace"
  }
]
```

organizations show

GET /v1/organizations/{id}

Retrieve a single organizations resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations show --id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "802315",
    "type": "organizations",
    "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f",
    "attributes": {
      "name": "Acme Accounting Co.",
      "stripe_customer_id": "cus_IRj1sm3XldF08O",
      "feature_flags": [
```

```
"user_pricing",
"manual_subscriptions"
],
"msp_feature_flags": [],
"managed_by_msp_id": null,
"owned_msp_id": null,
"anceled": false,
"anceled_at": null,
"will_cancel_at": null,
"address": "",
"trial_days": null,
"billing_address": null,
"billing_manual": false,
"billing_contact_name": "",
"billing_contact_phone": "",
"billing_contact_email": "",
"first_traffic_sent": "2020-11-23T19:43:01.948-05:00",
"gdpr": false,
"summary_email_enabled": false,
"summary_email_address": null,
"release_channel": "stable",
"sync_tools_auto_update": true,
"user_agents_auto_update": true,
"usage_alert_cents": null,
"unbounce_data": null,
"plan_details": null,
"update_payment": true,
"industry": "5",
"external_id": null,
"metadata": null,
"unique_id": "4f53f068-6636-4a6e-b226-00f7732a9b03",
"sso_enabled": false,
"allowlist_updated_at": "2025-01-09T15:46:38.201-05:00",
"blocklist_updated_at": null,
"send_uninstall_notifications_to_admin_users": false,
"user_agent_uninstall_notification": true,
"user_agent_uninstall_notification_recipient_emails": [
  "jane.smith@example.com"
],
"pin_protected_uninstall": true,
"fetch_onboard_details": true,
```



```
"active_psa_integration": false,  
"show_pii_rc_hostnames": false,  
"segment": "regular-2024-phase-one",  
"privacy_mode": "standard",  
"vpn_settings_organization": null,  
"entitlements_enabled": true,  
"role": "owner",  
"current_mrr": 0,  
"current_license": "",  
"show_billing": true,  
...
```

organizations update

PATCH /v1/organizations/{id}

Update an existing organizations resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 802315
--name A human-readable display name for this resource, e.g. --name "New Client Corp Updated"
--billing_contact_name Billing contact name, e.g. --billing_contact_name "Jane Smith"
--billing_contact_phone Billing contact phone, e.g. --billing_contact_phone +12025551234
--billing_contact_email Billing contact email, e.g. --billing_contact_email billing@company.com
--address The IP address (for ip-addresses) or MAC address (for mac-addresses), e.g. --address 203.0.113.5
--managed_by_msp_id Managing MSP ID, e.g. --managed_by_msp_id 8801
--show_pii_rc_hostnames Show PII roaming client hostnames, e.g. --show_pii_rc_hostnames false
--unique_id External unique identifier, e.g. --unique_id ext-client-001
--sku A product SKU code. Example: professional, e.g. --sku professional
--quantity Seat quantity, e.g. --quantity 25
--gdpr Enable GDPR mode, e.g. --gdpr true
--privacy_mode Privacy mode (standard / strict), e.g. --privacy_mode standard
--enable_cybersight Enable CyberSight, e.g. --enable_cybersight true
--vpn_settings_organization_attributes VPN settings JSON object, e.g. --vpn_settings_organization_attributes {"enabled":false}

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations update --id 802315 --name "New Client Corp Updated"
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 802315,  
  "type": "organizations",  
  "attributes": {  
    "name": "New Client Corp Updated",  
    "updated_at": "2025-06-01T10:00:00.000-04:00"  
  }  
}
```

organizations users-create

POST /v1/organizations/{organization_id}/users

Add a user to an organization.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--email` An email address. Example: `admin@company.com`, e.g. `--email admin@company.com`
`--first_name` The user's given (first) name, e.g. `--first_name "Jane"`
`--last_name` The user's family (last) name, e.g. `--last_name "Smith"`
`--phone` A phone number. Example: `+12025551234`, e.g. `--phone +12025551234`
`--role` The user's role. One of: `administrator`, `read_only`, `network_administrator`, `network_support`, `support`, e.g. `--role administrator`
`--organization_permission_ids` Array of permission IDs, e.g. `--organization_permission_ids ["perm-1","perm-2"]`
`--is_include_only_list` Include-only permission list, e.g. `--is_include_only_list false`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py organizations users-create \  
  --organization_id 802315 \  
  --email admin@company.com \  
  --first_name "Jane" \  
  --last_name "Smith" \  
  --phone +12025551234 \  
  --role administrator \  
  --organization_permission_ids ["perm-1","perm-2"] \  
  --is_include_only_list false
```

```
--last_name "Smith" \  
--role administrator
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 9999903,  
  "type": "users",  
  "attributes": {  
    "email": "admin@company.com",  
    "first_name": "Jane",  
    "last_name": "Smith",  
    "role": "administrator"  
  }  
}
```

organizations users-delete

DELETE /v1/organizations/{organization_id}/users/{id}

Remove a user from an organization.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 42618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py organizations users-delete --organization_id 802315 --id 42618
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

organizations users-list

GET /v1/organizations/{organization_id}/users

List users in an organization.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py organizations users-list --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{
  "users": [
    {
      "id": 42618,
      "name": "Jane Smith",
      "email": "jane.smith@example.com",
      "created_at": "2020-11-23T19:40:02.381-05:00",
      "updated_at": "2026-05-21T17:21:26.365-04:00",
    }
  ]
}
```

```

"auth0_sub": "auth0|5fbc5661dd161e00760459c7",
"first_name": "Jane",
"last_name": "Smith",
"phone": "+14035128790",
"email_verified": true,
"mfa_enabled": true,
"was_member_of_organization": true,
"last_sign_in_at": "2026-05-21T17:21:26.365-04:00",
"distributor_guid": null,
"feature_flags": [],
"distributor_id": null,
"transferred_at": null,
"metadata": {},
"uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
"must_reset_password": false,
"role": "owner",
"is_include_only_list": false,
"organization_permission_ids": []
},
{
  "id": 120904,
  "name": "Desktop C2",
  "email": "jdoe@example.com",
  "created_at": "2023-03-01T22:14:57.452-05:00",
  "updated_at": "2025-01-15T13:10:24.989-05:00",
  "auth0_sub": "auth0|640014afc22fe97a7cfdba6b",
  "first_name": "Jane",
  "last_name": "C2",
  "phone": "",
  "email_verified": false,
  "mfa_enabled": false,
  "was_member_of_organization": true,
  "last_sign_in_at": "2024-12-18T10:41:32.902-05:00",
  "distributor_guid": null,
  "feature_flags": [],
  "distributor_id": null,
  "transferred_at": null,
  "metadata": {},
  "uuid": "0186a050-d52c-7da0-b331-5de7b02fb5b7",
  "must_reset_password": false,
  "role": "super_administrator",

```



```
"is_include_only_list": false,  
"organization_permission_ids": []  
}  
]  
}
```

organizations users-resend-invite

POST /v1/organizations/{organization_id}/users/{id}/resend_invite

Resend invite to an org user.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 42618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py organizations users-resend-invite --organization_id 802315 --id 42618
```

RESPONSE

HTTP 200 OK

```
{
  "status": "ok",
  "message": "Invitation resent successfully"
}
```

organizations users-show

GET /v1/organizations/{organization_id}/users/{id}

Show an org user.

FLAGS

--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315

--id [required] The numeric ID of the resource to operate on, e.g. --id 42618

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py organizations users-show --organization_id 802315 --id 42618
```

RESPONSE

HTTP 200 OK

```
{
  "user": {
    "id": 42618,
    "name": "Jane Smith",
    "email": "jane.smith@example.com",
    "created_at": "2020-11-23T19:40:02.381-05:00",
    "updated_at": "2026-05-21T17:21:26.365-04:00",
```

```
"auth0_sub": "auth0|5fbc5661dd161e00760459c7",
"first_name": "Jane",
"last_name": "Smith",
"phone": "+14035128790",
"email_verified": true,
"mfa_enabled": true,
"was_member_of_organization": true,
"last_sign_in_at": "2026-05-21T17:21:26.365-04:00",
"distributor_guid": null,
"feature_flags": [],
"distributor_id": null,
"transferred_at": null,
"metadata": {},
"uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
"must_reset_password": false,
"role": "owner",
"is_include_only_list": false,
"organization_permission_ids": []
}
}
```

organizations users-update

PATCH /v1/organizations/{organization_id}/users/{id}

Update an org user.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 42618`
`--email` An email address. Example: `admin@company.com`, e.g. `--email admin@company.com`
`--first_name` The user's given (first) name, e.g. `--first_name "Jane"`
`--last_name` The user's family (last) name, e.g. `--last_name "Smith"`
`--phone` A phone number. Example: `+12025551234`, e.g. `--phone +12025551234`
`--role` The user's role. One of: `administrator`, `read_only`, `network_administrator`, `network_support`, `support`, e.g. `--role read_only`
`--organization_permission_ids` Array of permission IDs, e.g. `--organization_permission_ids ["perm-1","perm-2"]`
`--is_include_only_list` Include-only permission list, e.g. `--is_include_only_list false`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py organizations users-update \  
  --organization_id 802315 \  
  --email admin@company.com \  
  --first_name Jane \  
  --last_name Smith \  
  --phone +12025551234 \  
  --role read_only \  
  --organization_permission_ids ["perm-1","perm-2"] \  
  --is_include_only_list false
```

```
--id 42618 \  
--role read_only
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 42618,  
  "type": "users",  
  "attributes": {  
    "role": "read_only",  
    "updated_at": "2025-06-01T10:00:00.000-04:00"  
  }  
}
```

policies add-allowed-application

POST /v1/policies/{id}/add_allowed_application

Allow an application on a policy.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--name [required] A human-readable display name for this resource, e.g. --name TikTok
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies add-allowed-application --id 285109 --name TikTok
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "allow_applications": [
      "TikTok"
    ]
  }
}
```

```
]
}
}
```


policies add-blacklist-category

POST /v1/policies/{id}/add_blacklist_category

Block a category on a policy.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--category_id [required] The numeric ID of a DNS filtering category. Use 'categories list' to see all IDs, e.g. --category_id 2
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies add-blacklist-category --id 285109 --category_id 2
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "blacklist_categories": [
```

```
    2  
    1  
  }  
}
```

policies add-blacklist-domain

POST /v1/policies/{id}/add_blacklist_domain

Block a domain on a policy.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 285109`
`--domain` [required] A fully-qualified domain name. Example: `malware.example.com`, e.g. `--domain malware.example.com`
`--note` The note text to attach to this domain on the resource, e.g. `--note "Flagged by threat intel"`
`--include_relationships` Include related objects (org, networks) in the response. Default: `true`, e.g. `--include_relationships true`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies add-blacklist-domain \  
  --id 285109 \  
  --domain malware.example.com \  
  --note "Flagged by threat intel"
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 285109,  
  "type": "policies",  
  "attributes": {  
    "blacklist_domains": [  
      "malware.example.com"  
    ]  
  }  
}
```

policies add-blocked-application

POST /v1/policies/{id}/add_blocked_application

Block an application on a policy.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--name [required] A human-readable display name for this resource, e.g. --name TikTok
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies add-blocked-application --id 285109 --name TikTok
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "block_applications": [
      "TikTok"
    ]
  }
}
```

```
]
}
}
```

policies add-whitelist-domain

POST /v1/policies/{id}/add_whitelist_domain

Allowlist a domain on a policy.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 285109`
`--domain` [required] A fully-qualified domain name. Example: `malware.example.com`, e.g. `--domain internal.corp.com`
`--note` The note text to attach to this domain on the resource, e.g. `--note "Known C2 infrastructure"`
`--include_relationships` Include related objects (org, networks) in the response. Default: `true`, e.g. `--include_relationships true`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies add-whitelist-domain --id 285109 --domain internal.corp.com
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
```

```
"attributes": {  
  "whitelist_domains": [  
    "internal.corp.com"  
  ]  
}
```


policies application

GET /v1/policies/application

List policies with application filtering for a specific application..

FLAGS

--application_id [required] Application ID, e.g. --application_id 1
--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315
--name A human-readable display name for this resource, e.g. --name "Guest WiFi"
--policy_ids An array of policy IDs to assign. Example: ["285109","331207"], e.g. --policy_ids ["285109","331207"]

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies application
```

RESPONSE

HTTP 404 Not Found

HTTP 404 is expected when the application policy feature is not enabled on this account.

HTTP 404 is expected when the application policy feature is not enabled on this account.

{

```
"error": "Unable to find the object that you requested."  
}
```

policies application-update

POST /v1/policies/application_update

Update application allow/block policy assignments..

FLAGS

--application_id [required] Application ID, e.g. --application_id 1
--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315
--allow_policies [required] Array of policy IDs to allow this app on, e.g. --allow_policies ["item1","item2"]
--block_policies [required] Array of policy IDs to block this app on, e.g. --block_policies ["item1","item2"]

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies application-update
```

RESPONSE

HTTP 200 OK

```
{
```

```
"status": "ok"  
}
```

policies bulk-add-allowlist

POST /v1/policies/bulk/add_allowlist_domains

Bulk add allowlist domains.

FLAGS

`--policy_ids` [required] An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids '["285109","331207"]'`
`--domains` [required] An array of domain names. Example: `["evil.com","malware.net"]`, e.g. `--domains '["safe.com","trusted.org"]'`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`
`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`
`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies bulk-add-allowlist \  
  --policy_ids '["285109","331207"]' \  
  --domains '["safe.com","trusted.org"]'
```

RESPONSE

HTTP 200 OK

```
{  
  "status": "ok",  
  "updated_policies": [  
    285109,
```

331207

]

}

policies bulk-add-blocklist

POST /v1/policies/bulk/add_blocklist_domains

Bulk add blocklist domains.

FLAGS

`--policy_ids` [required] An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids '["285109","331207"]'`
`--domains` [required] An array of domain names. Example: `["evil.com","malware.net"]`, e.g. `--domains '["evil.com","malware.net"]'`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`
`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`
`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies bulk-add-blocklist \  
  --policy_ids '["285109","331207"]' \  
  --domains '["evil.com","malware.net"]'
```

RESPONSE

HTTP 200 OK

```
{  
  "status": "ok",  
  "updated_policies": [  
    285109,
```

331207

]

}

policies bulk-remove-allowlist

POST /v1/policies/bulk/remove_allowlist_domains

Bulk remove allowlist domains.

FLAGS

`--policy_ids` [required] An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids '["285109"]'`
`--domains` [required] An array of domain names. Example: `["evil.com","malware.net"]`, e.g. `--domains '["safe.com"]'`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`
`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`
`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies bulk-remove-allowlist \  
  --policy_ids '["285109"]' \  
  --domains '["safe.com"]'
```

RESPONSE

HTTP 200 OK

```
{  
  "status": "ok",  
  "updated_policies": [  
    285109
```

```
]
}
```

policies bulk-remove-blocklist

POST /v1/policies/bulk/remove_blocklist_domains

Bulk remove blocklist domains.

FLAGS

`--policy_ids` [required] An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids '["285109"]'`
`--domains` [required] An array of domain names. Example: `["evil.com","malware.net"]`, e.g. `--domains '["evil.com"]'`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`
`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`
`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies bulk-remove-blocklist \  
  --policy_ids '["285109"]' \  
  --domains '["evil.com"]'
```

RESPONSE

HTTP 200 OK

```
{  
  "status": "ok",  
  "updated_policies": [  
    285109
```

```
]
}
```

policies create

POST /v1/policies

Create a new policies resource.

FLAGS

--name [required] A human-readable display name for this resource, e.g. --name "Guest WiFi"

--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315

--allow_unknown_domains Allow domains not yet classified by DNSFilter. Default: false, e.g. --allow_unknown_domains true

--google_safesearch Force Google SafeSearch on for all users on this policy, e.g. --google_safesearch true

--bing_safe_search Force Bing SafeSearch on for all users on this policy, e.g. --bing_safe_search true

--duck_duck_go_safe_search Force DuckDuckGo SafeSearch on for all users on this policy, e.g. --duck_duck_go_safe_search true

--ecosia_safesearch Force Ecosia SafeSearch on for all users on this policy, e.g. --ecosia_safesearch true

--yandex_safe_search Force Yandex SafeSearch on for all users on this policy, e.g. --yandex_safe_search true

--youtube_restricted Enable YouTube restricted mode for all users on this policy, e.g. --youtube_restricted true

--youtube_restricted_level YouTube restriction level. One of: strict, none, e.g. --youtube_restricted_level strict

--interstitial Show an interstitial warning page before blocked domains, e.g. --interstitial true

--allow_list_only Block all domains except those explicitly allowlisted. Default: false, e.g. --allow_list_only false

--is_global_policy Mark this policy as a global (default) policy for the organization, e.g. --is_global_policy false

--policy_ip_id Associated policy IP ID, e.g. --policy_ip_id 7701

--whitelist_domains Domains to pre-populate on the allowlist. JSON array of strings, e.g. --whitelist_domains ["safe.com","trusted.org"]

--blacklist_domains Domains to pre-populate on the blocklist. JSON array of strings, e.g. --blacklist_domains ["evil.com","malware.net"]

--blacklist_categories Category IDs to pre-populate on the blocklist. JSON array of

integers, e.g. `--blacklist_categories ["2","14","66"]`
`--allow_applications` Application names to pre-populate on the allowed-applications list, e.g. `--allow_applications ["Slack","Zoom"]`
`--block_applications` Application names to pre-populate on the blocked-applications list, e.g. `--block_applications ["TikTok","Instagram"]`
`--lock_version` Optimistic lock version, e.g. `--lock_version 23`
`--include_relationships` Include related objects (org, networks) in the response. Default: true, e.g. `--include_relationships true`
`--append_domains` If true, append to existing allow/block lists instead of replacing them, e.g. `--append_domains true`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies create \  
  --name "Guest WiFi" \  
  --organization_id 802315 \  
  --allow_unknown_domains true \  
  --google_safesearch true \  
  --youtube_restricted true \  
  --youtube_restricted_level strict
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 1501234,
```

```
"type": "policies",
"attributes": {
  "name": "Guest WiFi",
  "organization_id": 802315,
  "google_safesearch": true,
  "youtube_restricted": true,
  "allow_unknown_domains": true,
  "created_at": "2025-06-01T10:00:00.000-04:00"
}
}
```

policies delete

DELETE /v1/policies/{id}

Permanently delete a policies resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies delete --id 285109
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

policies list

GET /v1/policies

Retrieve a paginated list of policies resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1267725",
      "type": "policies",
      "uuid": "0196b695-55db-7533-8477-87605150b8bd",
      "attributes": {
        "name": "aeraer",
```

```
"organization_id": 802315,
"whitelist_domains": null,
"blacklist_domains": null,
"blacklist_categories": [
  56,
  69
],
"allow_unknown_domains": true,
"google_safesearch": false,
"youtube_restricted": false,
"policy_ip_id": null,
"interstitial": false,
"bing_safe_search": false,
"duck_duck_go_safe_search": false,
"ecosia_safesearch": false,
"youtube_restricted_level": "none",
"disable_iwf_blocking": false,
"friendly_wifi_certified": false,
"friendly_wifi_certified_at": null,
"deleted_at": null,
"yandex_safe_search": false,
"lock_version": 0,
"is_global_policy": false,
"can_edit": true,
"allow_applications": null,
"block_applications": null,
"allow_notes": {},
"block_notes": {},
"allowlist_updated_at": null,
"blocklist_updated_at": null,
"allow_list_only": false
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  }
},
"networks": {
```

```
"data": []  
},  
"mac_addresses": {  
  "data": []  
},  
"scheduled_policies": {  
  "data": []  
},  
"network_subnets": {  
  "data": []  
},  
...
```

policies list-all

GET /v1/policies/all

Retrieve all policies resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py policies list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1486063",
      "type": "policies",
      "uuid": "019d1fdc-6d91-7efd-8a8b-64de8ff9df02",
      "attributes": {
        "name": "Testing NXDomain blocks",
        "organization_id": 802315,
        "whitelist_domains": [],
        "blacklist_domains": [
          "edu.co",
          "adfadfqrssqrszzzz.com",
          "reddit.com",
          "xnxx.com"
        ],
        "blacklist_categories": [
          56,
          69
        ],
        "allow_unknown_domains": true,
        "google_safesearch": false,
        "youtube_restricted": false,
        "policy_ip_id": null,
        "interstitial": false,

```

```
"bing_safe_search": false,
"duck_duck_go_safe_search": false,
"ecosia_safesearch": false,
"youtube_restricted_level": "none",
"disable_iwf_blocking": false,
"friendly_wifi_certified": false,
"friendly_wifi_certified_at": null,
"deleted_at": null,
"yandex_safe_search": false,
"lock_version": 2,
"is_global_policy": false,
"can_edit": true,
"allow_applications": null,
"block_applications": [],
"allow_notes": {},
"block_notes": {},
"allowlist_updated_at": "2026-03-24T08:40:32.913-04:00",
"blocklist_updated_at": "2026-03-24T08:49:55.424-04:00",
"allow_list_only": false
},
"relationships": {
  "organization": {
    "data": {
      "id": "802315",
      "type": "organizations",
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"
    }
  }
},
"networks": {
  "data": [
    {
      "id": "736401",
      "type": "networks",
      "uuid": "0175f7b5-7300-7072-b35a-25d10d0d0f3e"
    }
  ],
  ...
}
```

policies permissive-mode

GET /v1/policies/{id}/permissive_mode

Show permissive mode status.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies permissive-mode --id 285109
```

RESPONSE

HTTP 200 OK

```
{
  "permissive_mode": true
}
```

policies remove-allowed-application

POST /v1/policies/{id}/remove_allowed_application

Remove an allowed application.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--name [required] A human-readable display name for this resource, e.g. --name TikTok
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies remove-allowed-application --id 285109 --name TikTok
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "allow_applications": []
  }
}
```

```
}  
}
```


policies remove-blacklist-category

POST /v1/policies/{id}/remove_blacklist_category

Unblock a category.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--category_id [required] The numeric ID of a DNS filtering category. Use 'categories list' to see all IDs, e.g. --category_id 2
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies remove-blacklist-category --id 285109 --category_id 2
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "blacklist_categories": []
  }
}
```

```
}  
}
```

policies remove-blacklist-domain

POST /v1/policies/{id}/remove_blacklist_domain

Unblock a domain.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--domain [required] A fully-qualified domain name. Example: malware.example.com, e.g. --domain malware.example.com
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies remove-blacklist-domain \  
  --id 285109 \  
  --domain malware.example.com
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 285109,  
  "type": "policies",
```

```
"attributes": {  
  "blacklist_domains": []  
}  
}
```

policies remove-blocked-application

POST /v1/policies/{id}/remove_blocked_application

Unblock an application.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--name [required] A human-readable display name for this resource, e.g. --name TikTok
--include_relationships Include related objects (org, networks) in the response. Default: true, e.g. --include_relationships true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies remove-blocked-application --id 285109 --name TikTok
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "block_applications": []
  }
}
```

```
}  
}
```

policies remove-whitelist-domain

POST /v1/policies/{id}/remove_whitelist_domain

Remove a domain from allowlist.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 285109`
`--domain` [required] A fully-qualified domain name. Example: `malware.example.com`, e.g. `--domain internal.corp.com`
`--include_relationships` Include related objects (org, networks) in the response. Default: `true`, e.g. `--include_relationships true`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies remove-whitelist-domain --id 285109 --domain internal.corp.com
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "type": "policies",
  "attributes": {
    "whitelist_domains": []
  }
}
```

```
}  
}
```


policies set-permissive-mode

POST /v1/policies/{id}/permissive_mode

Enable/disable permissive mode.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--permissive_mode [required] true to enable permissive mode, false to disable, e.g. --permissive_mode true

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies set-permissive-mode --id 285109
```

RESPONSE

HTTP 200 OK

```
{
  "id": 285109,
  "permissive_mode": true
}
```

policies show

GET /v1/policies/{id}

Retrieve a single policies resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policies show --id 285109
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "285109",
    "type": "policies",
    "uuid": "0175f7b5-722d-712b-9de0-747e3b297ec0",
    "attributes": {
      "name": "Block Adult Content",
      "organization_id": 802315,
      "whitelist_domains": [
```

```
"choctop.us"
],
"blacklist_domains": [
  "cname.vercel-dns.com",
  "instagram.com",
  "facebook.com",
  "fbcdn.net",
  "facebook.net",
  "cdninstagram.com",
  "debug.dnsfilter.com"
],
"blacklist_categories": [
  56,
  69,
  2,
  38
],
"allow_unknown_domains": true,
"google_safesearch": false,
"youtube_restricted": true,
"policy_ip_id": null,
"interstitial": true,
"bing_safe_search": true,
"duck_duck_go_safe_search": true,
"ecosia_safesearch": true,
"youtube_restricted_level": "moderate",
"disable_iwf_blocking": false,
"friendly_wifi_certified": false,
"friendly_wifi_certified_at": null,
"deleted_at": null,
"yandex_safe_search": true,
"lock_version": 23,
"is_global_policy": false,
"can_edit": true,
"allow_applications": null,
"block_applications": null,
"allow_notes": {},
"block_notes": {},
"allowlist_updated_at": "2025-02-04T13:35:08.429-05:00",
"blocklist_updated_at": "2025-05-15T14:15:40.883-04:00",
"allow_list_only": false
```

```
},  
"relationships": {  
  "organization": {  
    "data": {  
      "id": "802315",  
      "type": "organizations",  
      "uuid": "0175f7b1-705e-7f7e-ab71-9f6b84c7798f"  
    }  
  },  
  "networks": {  
    ...  
  }  
}
```

policies update

PATCH /v1/policies/{id}

Update an existing policies resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109
--name [required] A human-readable display name for this resource, e.g. --name "Guest WiFi Updated"
--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315
--allow_unknown_domains Allow domains not yet classified by DNSFilter. Default: false, e.g. --allow_unknown_domains true
--google_safesearch Force Google SafeSearch on for all users on this policy, e.g. --google_safesearch true
--bing_safe_search Force Bing SafeSearch on for all users on this policy, e.g. --bing_safe_search true
--duck_duck_go_safe_search Force DuckDuckGo SafeSearch on for all users on this policy, e.g. --duck_duck_go_safe_search true
--ecosia_safesearch Force Ecosia SafeSearch on for all users on this policy, e.g. --ecosia_safesearch true
--yandex_safe_search Force Yandex SafeSearch on for all users on this policy, e.g. --yandex_safe_search true
--youtube_restricted Enable YouTube restricted mode for all users on this policy, e.g. --youtube_restricted true
--youtube_restricted_level YouTube restriction level. One of: strict, none, e.g. --youtube_restricted_level strict
--interstitial Show an interstitial warning page before blocked domains, e.g. --interstitial true
--allow_list_only Block all domains except those explicitly allowlisted. Default: false, e.g. --allow_list_only false
--is_global_policy Mark this policy as a global (default) policy for the organization, e.g. --is_global_policy false
--policy_ip_id Associated policy IP ID, e.g. --policy_ip_id 7701
--whitelist_domains Domains to pre-populate on the allowlist. JSON array of strings, e.g. --whitelist_domains ["safe.com","trusted.org"]
--blacklist_domains Domains to pre-populate on the blocklist. JSON array of strings, e.g. --blacklist_domains ["evil.com","malware.net"]

`--blacklist_categories` Category IDs to pre-populate on the blacklist. JSON array of integers, e.g. `--blacklist_categories ["2","14","66"]`
`--allow_applications` Application names to pre-populate on the allowed-applications list, e.g. `--allow_applications ["Slack","Zoom"]`
`--block_applications` Application names to pre-populate on the blocked-applications list, e.g. `--block_applications ["TikTok","Instagram"]`
`--lock_version` Optimistic lock version, e.g. `--lock_version 23`
`--include_relationships` Include related objects (org, networks) in the response. Default: true, e.g. `--include_relationships true`
`--append_domains` If true, append to existing allow/block lists instead of replacing them, e.g. `--append_domains true`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py policies update \  
  --id 285109 \  
  --name "Guest WiFi Updated" \  
  --organization_id 1 \  
  --interstitial true
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 285109,  
  "type": "policies",
```

```
"attributes": {  
  "name": "Guest WiFi Updated",  
  "organization_id": 802315,  
  "interstitial": true,  
  "updated_at": "2025-06-01T10:00:00.000-04:00"  
}  
}
```

policy-ips list

GET /v1/policy_ips

Retrieve a paginated list of policy-ips resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policy-ips list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "1",
      "type": "policy_ips",
      "uuid": "0154c04f-ebdb-759f-9d72-52d8996a31a6",
      "attributes": {
        "friendly_id": 1,
```



```
"primary_address": "103.247.36.101",
"secondary_address": "103.247.37.101"
},
{
  "id": "2",
  "type": "policy_ips",
  "uuid": "0154c050-0d48-76df-97cb-7a061b884a2d",
  "attributes": {
    "friendly_id": 2,
    "primary_address": "103.247.36.102",
    "secondary_address": "103.247.37.102"
  }
},
{
  "id": "3",
  "type": "policy_ips",
  "uuid": "0154c050-283b-7ec7-ad0c-41cc3bd97fda",
  "attributes": {
    "friendly_id": 3,
    "primary_address": "103.247.36.103",
    "secondary_address": "103.247.37.103"
  }
},
{
  "id": "4",
  "type": "policy_ips",
  "uuid": "0154c050-4443-78c2-9fff-0ac1e369f88d",
  "attributes": {
    "friendly_id": 4,
    "primary_address": "103.247.36.104",
    "secondary_address": "103.247.37.104"
  }
},
{
  "id": "5",
  "type": "policy_ips",
  "uuid": "0154c050-64cd-7d0f-a0a7-e0b8ea478a81",
  "attributes": {
    "friendly_id": 5,
    "primary_address": "103.247.36.105",
```

```
"secondary_address": "103.247.37.105"
}
},
{
  "id": "6",
  "type": "policy_ips",
  "uuid": "0154c050-7c40-7f91-a7ab-8ecfb3d88bfa",
  "attributes": {
    "friendly_id": 6,
    "primary_address": "103.247.36.106",
    "secondary_address": "103.247.37.106"
  }
  ...
}
```

policy-ips show

GET /v1/policy_ips/{id}

Retrieve a single policy-ips resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id pp-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py policy-ips show --id pp-001
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "1",
    "type": "policy_ips",
    "uuid": "0154c04f-ebdb-759f-9d72-52d8996a31a6",
    "attributes": {
      "friendly_id": 1,
      "primary_address": "103.247.36.101",
      "secondary_address": "103.247.37.101"
    }
  }
}
```

```
}  
}  
}
```

psa-integrations redirect-link

GET /v1/psa_integrations/redirect_link

Get PSA integration redirect link.

EXAMPLE COMMAND

```
python dnscli.py psa-integrations redirect-link
```

RESPONSE

HTTP 404 Not Found

HTTP 404 is expected when no PSA integration is configured on this account.

HTTP 404 is expected when no PSA integration is configured on this account.

```
{  
  "error": "Unable to find the object that you requested."  
}
```

scheduled-policies create

POST /v1/scheduled_policies

Create a new scheduled-policies resource.

FLAGS

`--name` [required] A human-readable display name for this resource, e.g. `--name "School Hours"`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--policy_ids` An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids '["285109"]'`
`--timezone` IANA timezone string. Example: `America/Denver`, e.g. `--timezone America/Denver`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnscli.py scheduled-policies create \  
  --name "School Hours" \  
  --organization_id 802315 \  
  --policy_ids '["285109"]' \  
  --timezone America/Denver
```

RESPONSE

HTTP 201 Created

```
{
  "id": 99102,
  "type": "scheduled_policies",
  "attributes": {
    "name": "School Hours",
    "organization_id": 802315,
    "policy_ids": [
      285109
    ],
    "timezone": "America/Denver"
  }
}
```

scheduled-policies delete

DELETE /v1/scheduled_policies/{id}

Permanently delete a scheduled-policies resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 71203`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-policies delete --id 71203
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

scheduled-policies list

GET /v1/scheduled_policies

Retrieve a paginated list of scheduled-policies resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-policies list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/scheduled_policies?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",
    "first": "https://api.dnsfilter.com/v1/scheduled_policies?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",
    "prev": null,
```

```
"next": null,  
"last": "https://api.dnsfilter.com/v1/scheduled_policies?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2"  
}  
}
```

scheduled-policies list-all

GET /v1/scheduled_policies/all

Retrieve all scheduled-policies resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py scheduled-policies list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/scheduled_policies/all?
page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "first": "https://api.dnsfilter.com/v1/scheduled_policies/all?
page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "prev": null,
    "next": null,
    "last": "https://api.dnsfilter.com/v1/scheduled_policies/all?
page%5Bnumber%5D=1&page%5Bsize%5D=1500"
  }
}
```

scheduled-policies show

GET /v1/scheduled_policies/{id}

Retrieve a single scheduled-policies resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 71203

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-policies show --id 71203
```

RESPONSE

HTTP 200 OK

```
{
  "data": [],
  "links": {
    "self": "https://api.dnsfilter.com/v1/scheduled_policies?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=1",
    "first": "https://api.dnsfilter.com/v1/scheduled_policies?
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=1",
    "prev": null,
    "next": null,
```

```
"last": "https://api.dnsfilter.com/v1/scheduled_policies?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=1"  
}  
}
```

scheduled-policies update

PATCH /v1/scheduled_policies/{id}

Update an existing scheduled-policies resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 71203`
`--name` A human-readable display name for this resource, e.g. `--name "School Hours Updated"`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--policy_ids` An array of policy IDs to assign. Example: `["285109","331207"]`, e.g. `--policy_ids ["285109","331207"]`
`--timezone` IANA timezone string. Example: `America/Denver`, e.g. `--timezone America/Chicago`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-policies update \  
  --id 71203 \  
  --name "School Hours Updated" \  
  --timezone America/Chicago
```

RESPONSE

HTTP 200 OK

```
{  
  "id": 71203,  
  "type": "scheduled_policies",  
  "attributes": {  
    "name": "School Hours Updated",  
    "timezone": "America/Chicago"  
  }  
}
```

scheduled-reports create

POST /v1/scheduled_reports

Create a new scheduled-reports resource.

FLAGS

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--frequency` Report frequency. One of: daily, weekly, monthly, e.g. `--frequency weekly`
`--day_of_week` Day of week for weekly reports. 0=Sunday, 1=Monday ... 6=Saturday, e.g. `--day_of_week 1`
`--include_threat_summary` Include the threat summary section in the report. true or false, e.g. `--include_threat_summary true`
`--include_content_category_summary` Include the content category summary section. true or false, e.g. `--include_content_category_summary true`
`--content_categories_show_count` Number of categories to show, e.g. `--content_categories_show_count 10`
`--send_to_dashboard_users` Send the report to all dashboard users. true or false, e.g. `--send_to_dashboard_users true`
`--scheduled_report_recipients` Array of recipient objects (JSON), e.g. `--scheduled_report_recipients [{"email":"admin@company.com"}]`
`--selected_sub_orgs` Array of sub-org IDs to include, e.g. `--selected_sub_orgs ["802316","491461"]`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-reports create \  
  --organization_id 802315 \  
  --frequency weekly \  
  --day_of_week 1 \  
  --include_threat_summary true \  
  --send_to_dashboard_users true
```

RESPONSE

HTTP 201 Created

```
{  
  "id": 9901,  
  "type": "scheduled_reports",  
  "attributes": {  
    "organization_id": 802315,  
    "frequency": "weekly",  
    "day_of_week": "1",  
    "include_threat_summary": true  
  }  
}
```

scheduled-reports delete

DELETE /v1/scheduled_reports/{id}

Permanently delete a scheduled-reports resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 9901`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-reports delete --id 9901
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

scheduled-reports list

GET /v1/scheduled_reports

Retrieve a paginated list of scheduled-reports resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25
--organization_id [required] The numeric ID of the organization that owns this resource, e.g. --organization_id 802315

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-reports list --page 1 --per_page 25 --organization_id 802315
```

RESPONSE

HTTP 200 OK

```
{  
  "data": []  
}
```

scheduled-reports preview-create

POST /v1/scheduled_report_previews

Create a report preview.

FLAGS

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--include_threat_summary` Include the threat summary section in the report. true or false, e.g. `--include_threat_summary true`
`--include_content_category_summary` Include the content category summary section. true or false, e.g. `--include_content_category_summary true`
`--content_categories_show_count` Categories to show, e.g. `--content_categories_show_count 10`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-reports preview-create \  
  --organization_id 802315 \  
  --include_threat_summary true
```

RESPONSE

HTTP 202 Accepted

```
{  
  "id": "preview-001",  
  "status": "pending",  
  "created_at": "2025-06-01T10:00:00.000-04:00"  
}
```

scheduled-reports preview-show

GET /v1/scheduled_report_previews/{id}

Show a report preview.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id preview-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-reports preview-show --id preview-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "preview-001",
  "status": "completed",
  "download_url": "https://api.dnsfilter.com/reports/preview-001.pdf",
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:20.000-04:00"
}
```

scheduled-reports show

GET /v1/scheduled_reports/{id}

Retrieve a single scheduled-reports resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 9901

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py scheduled-reports show --id 9901
```

RESPONSE

HTTP 200 OK

Empty response — this is the expected behaviour for this endpoint.

It indicates the operation succeeded but no data was returned.

```
{}
```

scheduled-reports update

PATCH /v1/scheduled_reports/{id}

Update an existing scheduled-reports resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 9901`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--frequency` Report frequency. One of: daily, weekly, monthly, e.g. `--frequency monthly`
`--day_of_week` Day of week for weekly reports. 0=Sunday, 1=Monday ... 6=Saturday, e.g. `--day_of_week 1`
`--include_threat_summary` Include the threat summary section in the report. true or false, e.g. `--include_threat_summary true`
`--include_content_category_summary` Include the content category summary section. true or false, e.g. `--include_content_category_summary true`
`--content_categories_show_count` Number of categories to show, e.g. `--content_categories_show_count 10`
`--send_to_dashboard_users` Send the report to all dashboard users. true or false, e.g. `--send_to_dashboard_users true`
`--scheduled_report_recipients` Array of recipient objects (JSON), e.g. `--scheduled_report_recipients [{"email":"admin@company.com"}]`
`--selected_sub_orgs` Array of sub-org IDs to include, e.g. `--selected_sub_orgs ["802316","491461"]`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnscli.py scheduled-reports update --id 9901 --frequency monthly
```

RESPONSE

HTTP 200 OK

```
{
  "id": 9901,
  "type": "scheduled_reports",
  "attributes": {
    "frequency": "monthly",
    "updated_at": "2025-06-01T10:00:00.000-04:00"
  }
}
```

traffic-reports qps

GET /v1/traffic_reports/qps

Queries per second.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnscli.py traffic-reports qps --start_date 2025-01-01 --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "network_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "qps": 0.9,
        "qps_networks": 0.9,
        "qps_proxies": 0,
        "qps_agents": 0
      }
    ]
  }
}
```

traffic-reports qps-active-agents

GET /v1/traffic_reports/qps_active_agents

QPS for active agents.

FLAGS

--from [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. --from 10.0.1.0

--to [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. --to 10.0.1.255

--organization_id The numeric ID of the organization that owns this resource, e.g. --organization_id 802315

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports qps-active-agents \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 400 Bad Request

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

```
{  
  "error": "time period is greater than 20 minutes (1442)"  
}
```

traffic-reports qps-active-collections

GET /v1/traffic_reports/qps_active_collections

QPS for active collections.

FLAGS

`--from` [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. `--from 10.0.1.0`

`--to` [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. `--to 10.0.1.255`

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`

`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`

`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports qps-active-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 400 Bad Request

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

```
{  
  "error": "time period is greater than 20 minutes (1442)"  
}
```

traffic-reports qps-active-organizations

GET /v1/traffic_reports/qps_active_organizations

QPS for active organizations.

FLAGS

`--from` [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. `--from 10.0.1.0`

`--to` [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. `--to 10.0.1.255`

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`

`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`

`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports qps-active-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 400 Bad Request

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

```
{  
  "error": "time period is greater than 20 minutes (1442)"  
}
```

traffic-reports qps-active-users

GET /v1/traffic_reports/qps_active_users

QPS for active users.

FLAGS

`--from` [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. `--from 10.0.1.0`

`--to` [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. `--to 10.0.1.255`

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`

`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`

`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports qps-active-users \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 400 Bad Request

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

HTTP 400 is expected when the time range is too large. QPS endpoints require a narrow real-time window.

```
{  
  "error": "time period is greater than 20 minutes (1442)"  
}
```

traffic-reports query-logs

GET /v1/traffic_reports/query_logs

Query log export.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports query-logs --start_date 2025-01-01 --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "user_ids": [],
    "collection_ids": [],
    "values": [
      {
        "id": 0,
        "time": "2026-06-02 15:27:12.990",
        "fqdn": "e2c47.gcp.gvt2.com",
        "networkid": 9710205,
        "request_address": "98.124.127.226",
        "domain": "gvt2.com",
        "result": "allowed",
        "threat": false,
        "categories": [
          40
        ],
        "server_id": 20214,
        "server_address": "103.247.36.36",
        "method": 20,
        "responsetime": 0,
        "agentid": null,
        "agentname": null,
        "agenttype": null,
        "agent_type_raw": null,
        "local_ipv4_address": null,
        "local_ipv6_address": null,
        "local_user_id": null,
        "lan_device_name": null,
        "policy_id": 1486063,
        "policy_name": "Testing NXDomain blocks",
        "scheduled_policy_id": null,

```

```
"scheduled_policy_name": null,  
"collection_id": null,  
"collection_name": null,  
"collections": [  
  0  
],  
"collections_names": [],  
"mac_address": null,  
"application_id": null,  
"application_name": null,  
"application_category_id": null,  
"application_category_name": null,  
"protocol": "udp",  
"question_type": "A",  
"code": "NOERROR",  
"region": "WAS",  
"network_name": "Test",  
"categories_names": [  
  "Content Servers"  
],  
"method_name": "Allowed Per Category",  
...
```

traffic-reports top-agents

GET /v1/traffic_reports/top_agents

Top agents by query volume.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-agents --start_date 2025-01-01 --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 0,
    "total_count_networks": 0,
    "total_count_proxies": 0,
    "total_count_agents": 0
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "agent_ids": [],
    "agent_names": [],
    "values": [],
    "page": {
      "size": 10,
      "total": 0,
      "first": 0,
      "last": 0,
      "prev": 0,
      "next": 0,
      "self": 0
    }
  }
}
```


traffic-reports top-application-categories

GET /v1/traffic_reports/top_application_categories

Top application categories.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-application-categories \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 137012,
    "total_application_categories_sum": 20710
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "application_category_id": 13,
        "application_category_name": "Ecosystem Applications",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "organizations": [
          802315
        ],
        "organizations_names": [
          "Acme Accounting Co."
        ],
        "networks": [
          9710205
        ],
        "networks_names": [
          "Test"
        ],
        "agents": [],
        "agents_names": null,
        "collections": [],
```

```
"collections_names": [],
"users": [],
"users_names": null,
"users_logins": null,
"total": 13883,
"total_networks": 13883,
"total_proxies": 0,
"total_agents": 0
},
{
  "application_category_id": 6,
  "application_category_name": "Mail",
  "policies": [
    1486063
  ],
  "policies_names": [
    "Testing NXDomain blocks"
  ],
  "scheduled_policies": [],
  "scheduled_policies_names": null,
  "organizations": [
    ...
  ]
}
```

traffic-reports top-categories

GET /v1/traffic_reports/top_categories

Top DNS categories.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-categories \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 137012,
    "total_categories_sum": 177415
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "categoryid": 19,
        "category_id": 19,
        "category_name": "Information Technology",
        "methods": [
          28,
          4,
          20
        ],
        "methods_names": [
          "Official DNS Product Domain",
          "Open domain",
          "Allowed Per Category"
        ],
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "organizations": [
          802315
        ],
        "organizations_names": [
```

```
"Acme Accounting Co."
],
"networks": [
  9710205
],
"networks_names": [
  "Test"
],
"agents": [],
"agents_names": null,
"collections": [],
"collections_names": [],
"users": [],
"users_names": null,
"users_logins": null,
"total": 63060,
"total_networks": 63060,
"total_proxies": 0,
"total_agents": 0
},
{
  ...
```

traffic-reports top-collections

GET /v1/traffic_reports/top_collections

Top collections by query volume.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 0,
    "total_count_networks": 0,
    "total_count_proxies": 0,
    "total_count_agents": 0
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collection_ids": [],
    "collection_names": [],
    "values": [],
    "page": {
      "size": 10,
      "total": 0,
      "first": 0,
      "last": 0,
      "prev": 0,
      "next": 0,
      "self": 0
    }
  }
}
```


traffic-reports top-domains

GET /v1/traffic_reports/top_domains

Top queried domains.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-domains \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 136468,
    "total_count_networks": 136468,
    "total_count_proxies": 0,
    "total_count_agents": 0
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "domain": "apple.com",
        "allowed": [
          6,
          19,
          40,
          29
        ],
        "blocked": [],
        "methods": [
          20
        ],
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "organizations": [
          802315
        ],
        "organizations_names": [
```

```
"Acme Accounting Co."
],
"networks": [
  9710205
],
"networks_names": [
  "Test"
],
"agents": [],
"agents_names": null,
"collections": [],
"collections_names": [],
"users": [],
"users_names": null,
"users_logins": null,
"total": 9921,
"total_networks": 9921,
"total_proxies": 0,
"total_agents": 0,
"methods_names": [
  "Allowed Per Category"
]
...
```

traffic-reports top-networks

GET /v1/traffic_reports/top_networks

Top networks by query volume.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-networks \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 137022,
    "total_count_networks": 137022,
    "total_count_proxies": 0,
    "total_count_agents": 0
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [
      9710205
    ],
    "network_names": [
      "Test"
    ],
    "values": [
      {
        "total": 137022,
        "total_networks": 137022,
        "total_proxies": 0,
        "total_agents": 0,
        "network_id": 9710205,
        "network_name": "Test",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null
      }
    ],
    "page": {
```

```
"size": 10,  
"total": 1,  
"first": 1,  
"last": 1,  
"prev": null,  
"next": null,  
"self": 1  
}  
}  
}
```

traffic-reports top-organizations

GET /v1/traffic_reports/top_organizations

Top organizations by query volume.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 137023,
    "total_count_networks": 137023,
    "total_count_proxies": 0,
    "total_count_agents": 0
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "total": 137023,
        "total_networks": 137023,
        "total_proxies": 0,
        "total_agents": 0,
        "organization_id": 802315,
        "organization_name": "Acme Accounting Co.",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null
      }
    ],
    "page": {
      "size": 10,
      "total": 1,
      "first": 1,
      "last": 1,
      "prev": null,
      "next": null,
    }
  }
}
```



```
"self": 1  
}  
}  
}
```

traffic-reports top-organizations-requests

GET /v1/traffic_reports/top_organizations_requests

Top organizations by request count.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-organizations-requests \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "values": [
      {
        "organization_id": 802315,
        "total_requests": 137023,
        "organization_name": "Acme Accounting Co."
      }
    ],
    "page": {
      "size": 10,
      "total": 1,
      "first": 1,
      "last": 1,
      "prev": null,
      "next": null,
      "self": 1
    }
  }
}
```

traffic-reports top-users

GET /v1/traffic_reports/top_users

Top users by query volume.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports top-users --start_date 2025-01-01 --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "meta": {
    "total_count": 0,
    "total_count_networks": 0,
    "total_count_proxies": 0,
    "total_count_agents": 0
  },
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "user_ids": [],
    "user_names": [],
    "user_logins": [],
    "values": [],
    "page": {
      "size": 10,
      "total": 0,
      "first": 0,
      "last": 0,
      "prev": 0,
      "next": 0,
      "self": 0
    }
  }
}
```

traffic-reports total-applications-agents-stats

GET /v1/traffic_reports/total_applications_agents_stats

Total application stats by agent.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-applications-agents-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{  
  "data": {  
    "values": []  
  }  
}
```

traffic-reports total-applications-collections-stats

GET /v1/traffic_reports/total_applications_collections_stats

Total application stats by collection.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-applications-collections-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "values": [
      {
        "collection_id": 0,
        "collection_name": null,
        "total_requests": 20725,
        "allowed_requests": 20725,
        "blocked_requests": 0,
        "most_recent_bucket": "2026-06-02 15:15:00",
        "previous_total_requests": 15320
      }
    ]
  }
}
```

traffic-reports total-applications-networks-stats

GET /v1/traffic_reports/total_applications_networks_stats

Total application stats by network.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-applications-networks-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "values": [
      {
        "network_id": 9710205,
        "network_name": "Test",
        "total_requests": 20725,
        "allowed_requests": 20725,
        "blocked_requests": 0,
        "most_recent_bucket": "2026-06-02 15:15:00",
        "previous_total_requests": 15320
      }
    ]
  }
}
```

traffic-reports total-applications-organizations-stats

GET /v1/traffic_reports/total_applications_organizations_stats

Total application stats by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-applications-organizations-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "values": [
      {
        "organization_id": 802315,
        "organization_name": "Acme Accounting Co.",
        "total_requests": 20725,
        "allowed_requests": 20725,
        "blocked_requests": 0,
        "most_recent_bucket": "2026-06-02 15:15:00",
        "previous_total_requests": 15320
      }
    ]
  }
}
```

traffic-reports total-applications-stats

GET /v1/traffic_reports/total_applications_stats

Total application stats.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-applications-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "values": [
      {
        "application_id": 656,
        "application_name": "Microsoft",
        "application_category_id": 13,
        "application_category_name": "Ecosystem Applications",
        "total_requests": 6966,
        "allowed_requests": 6966,
        "blocked_requests": 0,
        "most_recent_bucket": "2026-06-02 15:15:00",
        "previous_total_requests": 6379
      },
      {
        "application_id": 116,
        "application_name": "Google",
        "application_category_id": 13,
        "application_category_name": "Ecosystem Applications",
        "total_requests": 5237,
        "allowed_requests": 5237,
        "blocked_requests": 0,
        "most_recent_bucket": "2026-06-02 15:15:00",
        "previous_total_requests": 3758
      },
      {
        "application_id": 484,
        "application_name": "Grammarly",
        "application_category_id": 2,
        "application_category_name": "Business",
        "total_requests": 3299,
        "allowed_requests": 3299,
        "blocked_requests": 0,
        "most_recent_bucket": "2026-06-02 15:15:00",
        "previous_total_requests": 1384
      },
      {
        "application_id": 389,
        "application_name": "Apple iTunes",
```

```
"application_category_id": 1016,  
"application_category_name": "BETA",  
"total_requests": 1505,  
"allowed_requests": 1505,  
"blocked_requests": 0,  
"most_recent_bucket": "2026-06-02 15:15:00",  
"previous_total_requests": 1539  
},  
{  
  "application_id": 105,  
  "application_name": "Amazon",  
  "application_category_id": 13,  
  "application_category_name": "Ecosystem Applications",  
  "total_requests": 1156,  
  "allowed_requests": 1156,  
  "blocked_requests": 0,  
  "most_recent_bucket": "2026-06-02 15:15:00",  
  "previous_total_requests": 454  
},  
{  
  "application_id": 614,  
  ...
```


traffic-reports total-applications-users-stats

GET /v1/traffic_reports/total_applications_users_stats

Total application stats by user.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-applications-users-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{  
  "data": {  
    "values": []  
  }  
}
```

traffic-reports total-categories

GET /v1/traffic_reports/total_categories

Total queries by category.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-categories \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "network_names": [],
    "category_ids": [
      -1,
      2,
      3,
      6,
      7,
      10,
      11,
      12,
      13,
      15,
      17,
      19,
      20,
      21,
      22,
      23,
      24,
      25,
      27,
      28,
      29,
      30,
      32,
      34,
      35,
      36,
      40,
      70,
```

```
71,  
74,  
75,  
76,  
77,  
82,  
83  
],  
"category_names": [  
  "Uncategorized",  
  "Adult Content",  
  "Advertising",  
  "Business",  
  "Webmail & Chat",  
  "Economy & Finance",  
  "Education & Self Help",  
  "Entertainment",  
  "Food & Recipes",  
  "Games",  
  "Health",  
  "Information Technology",  
  ...
```

traffic-reports total-categories-agents

GET /v1/traffic_reports/total_categories_agents

Total category stats by agent.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-categories-agents \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "agent_ids": [],
    "agent_names": [],
    "category_ids": [],
    "category_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-categories-collections

GET /v1/traffic_reports/total_categories_collections

Total category stats by collection.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-categories-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collection_ids": [],
    "collection_names": [],
    "category_ids": [],
    "category_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-categories-organizations

GET /v1/traffic_reports/total_categories_organizations

Total category stats by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-categories-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "category_ids": [
      -1,
      2,
      3,
      6,
      7,
      10,
      11,
      12,
      13,
      15,
      17,
      19,
      20,
      21,
      22,
      23,
      24,
      25,
      27,
      28,
      29,
      30,
      32,
      34,
      35,
      36,
      40,
      70,
      71,
      74,
```

```
75,  
76,  
77,  
82,  
83  
],  
"category_names": [  
  "Uncategorized",  
  "Adult Content",  
  "Advertising",  
  "Business",  
  "Webmail & Chat",  
  "Economy & Finance",  
  "Education & Self Help",  
  "Entertainment",  
  "Food & Recipes",  
  "Games",  
  "Health",  
  "Information Technology",  
  "Jobs & Careers",  
  "Media Sharing",  
  ...
```

traffic-reports total-categories-users

GET /v1/traffic_reports/total_categories_users

Total category stats by user.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-categories-users \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "user_ids": [],
    "user_names": [],
    "user_logins": [],
    "category_ids": [],
    "category_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-category-stats

GET /v1/traffic_reports/total_category_stats

Total category stats.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-category-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "total_requests": 136688,
    "allowed_requests": 136653,
    "blocked_requests": 35,
    "threat_requests": 547,
    "allowed_threat_requests": 547,
    "blocked_threat_requests": 0
  }
}
```


traffic-reports total-client-stats

GET /v1/traffic_reports/total_client_stats

Total client (agent) stats.

FLAGS

`--from` [required] The start IP address of the subnet range. Example: 10.0.1.0, e.g. `--from 10.0.1.0`
`--to` [required] The end IP address of the subnet range. Example: 10.0.1.255, e.g. `--to 10.0.1.255`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv` FILE Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv` FILE Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter` EXPR Keep matching rows only, e.g. `--filter status=active`
`--columns` a,b,c Limit table output to named columns, e.g. `--columns id,name`
`--org-id` ID Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-client-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 400 Bad Request

HTTP 400 is expected when the time range exceeds 20 minutes. This endpoint only accepts windows of 20 minutes or less.

HTTP 400 is expected when the time range exceeds 20 minutes. This endpoint only accepts windows of 20 minutes or less.

```
{  
  "error": "time period is greater than 20 minutes (1442)"  
}
```

traffic-reports total-collections

GET /v1/traffic_reports/total_collections

Total queries by collection.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "network_names": [],
    "collection_ids": [],
    "collection_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-collections-agents

GET /v1/traffic_reports/total_collections_agents

Total collection stats by agent.

FLAGS

--start_date [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. --start_date 2025-01-01
--end_date [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. --end_date 2025-01-31
--organization_id The numeric ID of the organization that owns this resource, e.g. --organization_id 802315
--network_id The numeric ID of the network to associate with, e.g. --network_id 736401
--limit Maximum number of results, e.g. --limit 1
--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-collections-agents \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "agent_ids": [],
    "agent_names": [],
    "collection_ids": [],
    "collection_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-collections-organizations

GET /v1/traffic_reports/total_collections_organizations

Total collection stats by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-collections-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collection_ids": [],
    "collection_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```


traffic-reports total-collections-users

GET /v1/traffic_reports/total_collections_users

Total collection stats by user.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-collections-users \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "user_ids": [],
    "user_names": [],
    "user_logins": [],
    "collection_ids": [],
    "collection_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-deployments

GET /v1/traffic_reports/total_deployments

Total deployments.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-deployments \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collections": 1,
    "user_agents": 5,
    "sync_tools": 1,
    "relays": 1,
    "users": 4
  }
}
```

traffic-reports total-domain-requests

GET /v1/traffic_reports/total_domain_requests

Total requests per domain.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-domain-requests \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "application_ids": [],
    "application_category_ids": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "total_requests": 79707,
        "allowed_requests": 79699,
        "blocked_requests": 8,
        "threat_requests": 344,
        "allowed_appaware_requests": 12149,
        "blocked_appaware_requests": 0
      }
    ]
  }
}
```

traffic-reports total-domain-stats

GET /v1/traffic_reports/total_domain_stats

Total domain stats.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-domain-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "total_requests": 136798,
    "allowed_requests": 136763,
    "blocked_requests": 35,
    "threat_requests": 547,
    "allowed_threat_requests": 547,
    "blocked_threat_requests": 0
  }
}
```


traffic-reports total-domains

GET /v1/traffic_reports/total_domains

Total unique domains queried.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-domains \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "network_names": [],
    "values": [
      {
        "domain": "Oedd968dc-frontier.amazon.com",
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 8,
        "total_networks": 8,
        "total_proxies": 0,
        "total_agents": 0
      },
      {
        "domain": "100.in-addr.arpa",
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 2,
```

```
"total_networks": 2,
"total_proxies": 0,
"total_agents": 0
},
{
  "domain": "104.in-addr.arpa",
  "bucket": "2026-06-02 00:00:00",
  "policies": [],
  "policies_names": [],
  "scheduled_policies": [],
  "scheduled_policies_names": null,
  "total": 0,
  "total_networks": 0,
  "total_proxies": 0,
  "total_agents": 0
},
{
  "domain": "108.in-addr.arpa",
  "bucket": "2026-06-02 00:00:00",
  "policies": [
    1486063
    ...
  ]
}
```

traffic-reports total-domains-collections

GET /v1/traffic_reports/total_domains_collections

Total domains by collection.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-domains-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collection_ids": [],
    "collection_names": [],
    "values": []
  }
}
```

traffic-reports total-domains-organizations

GET /v1/traffic_reports/total_domains_organizations

Total domains by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-domains-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "domain": "0edd968dc-frontier.amazon.com",
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 8,
        "total_networks": 8,
        "total_proxies": 0,
        "total_agents": 0
      },
      {
        "domain": "100.in-addr.arpa",
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 2,
        "total_networks": 2,
        "total_proxies": 0,

```

```
"total_agents": 0
},
{
  "domain": "104.in-addr.arpa",
  "bucket": "2026-06-02 00:00:00",
  "policies": [],
  "policies_names": [],
  "scheduled_policies": [],
  "scheduled_policies_names": null,
  "total": 0,
  "total_networks": 0,
  "total_proxies": 0,
  "total_agents": 0
},
{
  "domain": "108.in-addr.arpa",
  "bucket": "2026-06-02 00:00:00",
  "policies": [
    1486063
  ],
  "policies_names": [
    ...
  ]
}
```


traffic-reports total-domains-users

GET /v1/traffic_reports/total_domains_users

Total domains by user.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-domains-users \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "user_ids": [],
    "user_names": [],
    "user_logins": [],
    "values": []
  }
}
```

traffic-reports total-organizations-requests

GET /v1/traffic_reports/total_organizations_requests

Total requests by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-organizations-requests \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "application_ids": [],
    "application_category_ids": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "total_requests": 80052,
        "allowed_requests": 80044,
        "blocked_requests": 8,
        "threat_requests": 344,
        "allowed_appaware_requests": 12165,
        "blocked_appaware_requests": 0
      }
    ]
  }
}
```

traffic-reports total-organizations-stats

GET /v1/traffic_reports/total_organizations_stats

Total stats by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-organizations-stats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "total_requests": 137143,
    "allowed_requests": 137108,
    "blocked_requests": 35,
    "threat_requests": 547,
    "allowed_threat_requests": 547,
    "blocked_threat_requests": 0
  }
}
```

traffic-reports total-requests

GET /v1/traffic_reports/total_requests

Total DNS requests.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-requests \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "network_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 80057,
        "total_networks": 80057,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```


traffic-reports total-requests-agents

GET /v1/traffic_reports/total_requests_agents

Total requests by agent.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-requests-agents \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "agent_ids": [],
    "agent_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-requests-collections

GET /v1/traffic_reports/total_requests_collections

Total requests by collection.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-requests-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collection_ids": [],
    "collection_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-requests-geo

GET /v1/traffic_reports/total_requests_geo

Total requests by geography.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-requests-geo \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [
      9710205
    ],
    "network_names": [
      "Test"
    ],
    "values": [
      {
        "network_id": 9710205,
        "total_requests": 137157,
        "qps": 1.6,
        "network_name": "Test"
      }
    ]
  }
}
```

traffic-reports total-requests-organizations

GET /v1/traffic_reports/total_requests_organizations

Total requests by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-requests-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 80066,
        "total_networks": 80066,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```


traffic-reports total-requests-users

GET /v1/traffic_reports/total_requests_users

Total requests by user.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-requests-users \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "user_ids": [],
    "user_names": [],
    "user_logins": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-roaming-clients

GET /v1/traffic_reports/total_roaming_clients

Total roaming client stats.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-roaming-clients \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "os": "windows",
        "total": 4
      },
      {
        "os": "android",
        "total": 1
      }
    ]
  }
}
```

traffic-reports total-threats

GET /v1/traffic_reports/total_threats

Total threats blocked.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-threats \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "network_ids": [],
    "network_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 344,
        "total_networks": 344,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-threats-agents

GET /v1/traffic_reports/total_threats_agents

Total threats by agent.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-threats-agents \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "agent_ids": [],
    "agent_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```


traffic-reports total-threats-collections

GET /v1/traffic_reports/total_threats_collections

Total threats by collection.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-threats-collections \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "collection_ids": [],
    "collection_names": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-threats-organizations

GET /v1/traffic_reports/total_threats_organizations

Total threats by organization.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-threats-organizations \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [
          1486063
        ],
        "policies_names": [
          "Testing NXDomain blocks"
        ],
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 344,
        "total_networks": 344,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

traffic-reports total-threats-users

GET /v1/traffic_reports/total_threats_users

Total threats by user.

FLAGS

`--start_date` [required] Report start date in YYYY-MM-DD format. Example: 2025-01-01, e.g. `--start_date 2025-01-01`
`--end_date` [required] Report end date in YYYY-MM-DD format. Example: 2025-01-31, e.g. `--end_date 2025-01-31`
`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--limit` Maximum number of results, e.g. `--limit 1`
`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py traffic-reports total-threats-users \  
  --start_date 2025-01-01 \  
  --end_date 2025-01-31
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "organization_ids": [
      802315
    ],
    "organization_names": [
      "Acme Accounting Co."
    ],
    "user_ids": [],
    "user_names": [],
    "user_logins": [],
    "values": [
      {
        "bucket": "2026-06-02 00:00:00",
        "policies": [],
        "policies_names": null,
        "scheduled_policies": [],
        "scheduled_policies_names": null,
        "total": 0,
        "total_networks": 0,
        "total_proxies": 0,
        "total_agents": 0
      }
    ]
  }
}
```

user-agent-bulk-deletes counts

GET /v1/user_agent_bulk_deletes/counts

Count agents matching bulk delete criteria.

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-deletes counts
```

RESPONSE

HTTP 200 OK

```
{  
  "uninstall_and_delete": 3,  
  "delete": 9  
}
```

user-agent-bulk-deletes create

POST /v1/user_agent_bulk_deletes

Create a new user-agent-bulk-deletes resource.

FLAGS

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--ids` An array of resource IDs. Example: `["id1","id2"]`, e.g. `--ids '["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]'`
`--exclude_ids` An array of resource IDs to exclude from the operation, e.g. `--exclude_ids ["other-agent-uuid"]`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-deletes create \  
--ids '["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]'
```

RESPONSE

HTTP 202 Accepted

```
{  
  "id": "job-bd-001",  
  "status": "pending",
```



```
"created_at": "2025-06-01T10:00:00.000-04:00"  
}
```

user-agent-bulk-deletes show

GET /v1/user_agent_bulk_deletes/{id}

Retrieve a single user-agent-bulk-deletes resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id job-bd-001`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-deletes show --id job-bd-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-bd-001",
  "status": "completed",
  "deleted_count": 12,
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:10.000-04:00"
}
```

user-agent-bulk-updates counts

GET /v1/user_agent_bulk_updates/counts

Count agents matching bulk update criteria.

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-updates counts
```

RESPONSE

HTTP 200 OK

```
{  
  "undo_uninstall": 0,  
  "no_effect": 12  
}
```

user-agent-bulk-updates create

POST /v1/user_agent_bulk_updates

Create a new user-agent-bulk-updates resource.

FLAGS

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

`--ids` An array of resource IDs. Example: `["id1","id2"]`, e.g. `--ids ["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]`

`--exclude_ids` An array of resource IDs to exclude from the operation, e.g. `--exclude_ids ["other-agent-uuid"]`

`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`

`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`

`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`

`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`

`--friendly_name` The display name shown for this agent in the dashboard, e.g. `--friendly_name "Finance-Laptop"`

`--tags` Agent tags as a JSON array. Example: `["managed","finance"]`, e.g. `--tags ["managed","finance"]`

`--release_channels` Array of release channels, e.g. `--release_channels ["stable"]`

`--device_setting_attributes` Device settings JSON, e.g. `--device_setting_attributes {"auto_update":true}`

`--filtering_client_setting_attributes` Filter settings JSON, e.g. `--filtering_client_setting_attributes {"block_malware":true}`

`--vpn_settings_user_agent` VPN settings JSON, e.g. `--vpn_settings_user_agent {"enabled":true}`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-updates create \  
  --ids '["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]' \  
  --policy_id 285109
```

RESPONSE

HTTP 202 Accepted

```
{  
  "id": "job-bu-001",  
  "status": "pending",  
  "created_at": "2025-06-01T10:00:00.000-04:00"  
}
```

user-agent-bulk-updates has-mixed

POST /v1/user_agent_bulk_updates/has_mixed

Check for mixed values in bulk update selection.

FLAGS

--ids An array of resource IDs. Example: ["id1","id2"], e.g. **--ids** `'["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]'`

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. **--raw**

--json Output JSON to stdout (automatic when piped), e.g. **--json**

--to-csv FILE Write the response to a CSV file, e.g. **--to-csv** output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. **--from-csv** policies.csv

--template Print a blank CSV import template for this command and exit, e.g. **--template**

--filter EXPR Keep matching rows only, e.g. **--filter** status=active

--columns a,b,c Limit table output to named columns, e.g. **--columns** id,name

--org-id ID Override the stored organization ID for this call only, e.g. **--org-id** 802315

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-updates has-mixed \  
  --ids '["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]'
```

RESPONSE

HTTP 200 OK

```
{  
  "has_mixed": false,  
  "fields": {  
    "policy_id": false,  
    "network_id": true
```

```
}  
}
```

user-agent-bulk-updates show

GET /v1/user_agent_bulk_updates/{id}

Retrieve a single user-agent-bulk-updates resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id job-bu-001`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-bulk-updates show --id job-bu-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-bu-001",
  "status": "completed",
  "updated_count": 12,
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:09.000-04:00"
}
```


user-agent-cleanups create

POST /v1/user_agent_cleanups

Create a new user-agent-cleanups resource.

FLAGS

`--organization_id` The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`
`--organization_ids` [required] An array of organization IDs. Example: `["802315","802316"]`, e.g. `--organization_ids '["802315"]'`
`--inactive_for` [required] Number of days of inactivity after which agents are considered inactive, e.g. `--inactive_for 30`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-cleanups create \  
  --organization_ids '["802315"]' \  
  --inactive_for 30
```

RESPONSE

HTTP 202 Accepted

```
{  
  "id": "job-cleanup-001",
```

```
"status": "pending",  
"organization_ids": [  
  802315  
],  
"inactive_for": 30  
}
```

user-agent-cleanups show

GET /v1/user_agent_cleanups/{id}

Retrieve a single user-agent-cleanups resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id job-cleanup-001`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-cleanups show --id job-cleanup-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "job-cleanup-001",
  "status": "completed",
  "deleted_count": 4,
  "organization_ids": [
    802315
  ],
  "inactive_for": 30,
```

```
"completed_at": "2025-06-01T10:00:30.000-04:00"  
}
```

user-agent-cleanups update

PUT /v1/user_agent_cleanups/{id}

Update an existing user-agent-cleanups resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id job-cleanup-001
--start Set to true to start the cleanup job immediately, e.g. --start true
--inactive_for Number of days of inactivity after which agents are considered inactive, e.g.
--inactive_for 30

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv
policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --
template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-cleanups update \  
  --id job-cleanup-001 \  
  --start true \  
  --inactive_for 30
```

RESPONSE

HTTP 200 OK

```
{  
  "id": "job-cleanup-001",  
  "status": "running",
```

```
"start": true  
}
```

user-agent-csv-exports create

POST /v1/user_agent_csv_exports

Create a new user-agent-csv-exports resource.

FLAGS

`--organization_ids` An array of organization IDs. Example: ["802315","802316"], e.g. `--organization_ids '["802315"]'`
`--msp_id` MSP ID, e.g. `--msp_id 8801`
`--ids` An array of resource IDs. Example: ["id1","id2"], e.g. `--ids ["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]`
`--network_ids` Array of network IDs, e.g. `--network_ids ["736401","736402"]`
`--type` Note type. Example: block, e.g. `--type block`
`--search` Filter results by a text search string, e.g. `--search "laptop"`
`--name_search` Name search term, e.g. `--name_search "example"`
`--tags` Agent tags as a JSON array. Example: ["managed","finance"], e.g. `--tags ["managed","finance"]`
`--status` Filter by resource status, e.g. `--status active`
`--state` Agent state filter, e.g. `--state "Colorado"`
`--agent_state` Agent state, e.g. `--agent_state "example"`
`--traffic_received_last_15_mins` Only include agents active in last 15 min, e.g. `--traffic_received_last_15_mins true`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-csv-exports create --organization_ids '["802315"]'
```

RESPONSE

HTTP 202 Accepted

```
{  
  "id": "export-001",  
  "status": "pending",  
  "created_at": "2025-06-01T10:00:00.000-04:00"  
}
```


user-agent-csv-exports show

GET /v1/user_agent_csv_exports/{id}

Retrieve a single user-agent-csv-exports resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id export-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-csv-exports show --id export-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "export-001",
  "status": "completed",
  "download_url": "https://api.dnsfilter.com/exports/export-001.csv",
  "created_at": "2025-06-01T10:00:00.000-04:00",
  "completed_at": "2025-06-01T10:00:15.000-04:00"
}
```

user-agent-releases list

GET /v1/user_agent_releases

Retrieve a paginated list of user-agent-releases resources.

EXAMPLE COMMAND

```
python dnsfcli.py user-agent-releases list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "user_agent_releases": [
    {
      "agent_type": "windows",
      "architecture": "x86",
      "release_channels": [
        {
          "label": null,
          "value": "internal",
          "visible": false
        }
      ],
      "url": "https://download.dnsfilter.com/User_Agent/Windows/
DNSFilter_Agent_Setup_x86-1.10.3.0.msi",
      "version": "1.10.3",
      "white_label": false
    },
    {
      "agent_type": "windows",
      "architecture": "x86",
      "release_channels": [
        {
          "label": null,
          "value": "internal",
          "visible": false
        }
      ]
    }
  ]
}
```

```

    }
  ],
  "url": "https://download.dnsfilter.com/User_Agent/Windows/
DNS_Agent_Setup_x86-1.10.3.0.msi",
  "version": "1.10.3",
  "white_label": true
},
{
  "agent_type": "windows",
  "architecture": "x86",
  "release_channels": [
    {
      "label": "Production",
      "value": "stable",
      "visible": true
    }
  ],
  "url": "https://download.dnsfilter.com/User_Agent/Windows/
DNSFilter_Agent_Setup_x86-1.15.3.msi",
  "version": "1.15.3",
  "white_label": false
},
{
  "agent_type": "windows",
  "architecture": "x86",
  "release_channels": [
    {
      "label": "Production",
      "value": "stable",
      "visible": true
    }
  ],
  "url": "https://download.dnsfilter.com/User_Agent/Windows/
DNS_Agent_Setup_x86-1.15.3.msi",
  "version": "1.15.3",
  "white_label": true
},
{
  "agent_type": "windows",
  ...

```

user-agent-releases relay

GET /v1/user_agent_releases/relay

Show relay agent release info.

EXAMPLE COMMAND

```
python dnscli.py user-agent-releases relay
```

RESPONSE

HTTP 200 OK

```
{
  "user_agent_releases": [
    {
      "agent_type": "proxy",
      "architecture": "any",
      "release_channels": [
        {
          "label": "Production",
          "value": "stable",
          "visible": true
        }
      ],
      "url": "https://download.dnsfilter.com/download.dnsfilter.com/Relay/CloudRelay-2204LTS.zip",
      "version": "1.4.0",
      "white_label": false,
      "platform": "azure-cloud"
    },
    {
      "agent_type": "proxy",
      "architecture": "any",
      "release_channels": [
        {
          "label": "Production",
          "value": "stable",
```

```

    "visible": true
  }
],
  "url": "https://download.dnsfilter.com/download.dnsfilter.com/Relay/
HyperVRelay-2204LTS.vhdx",
  "version": "1.4.0",
  "white_label": false,
  "platform": "hyper-v"
},
{
  "agent_type": "proxy",
  "architecture": "any",
  "release_channels": [
    {
      "label": "Production",
      "value": "stable",
      "visible": true
    }
  ],
  "url": "https://download.dnsfilter.com/download.dnsfilter.com/Relay/
VirtualBoxRelay-2204LTS.ova",
  "version": "1.4.0",
  "white_label": false,
  "platform": "virtual-box"
},
{
  "agent_type": "proxy",
  "architecture": "any",
  "release_channels": [
    {
      "label": "Production",
      "value": "stable",
      "visible": true
    }
  ],
  "url": "https://download.dnsfilter.com/download.dnsfilter.com/Relay/VMWARE-
Relay22.04LTS.ova",
  "version": "1.4.0",
  "white_label": false,
  ...

```

user-agents counts

GET /v1/user_agents/counts

Agent counts.

EXAMPLE COMMAND

```
python dnsfcli.py user-agents counts
```

RESPONSE

HTTP 200 OK

```
{
  "protected": 5,
  "unprotected": 0,
  "bypassed": 0,
  "pending_uninstall": 0,
  "uninstalled": 7,
  "all": 12
}
```

user-agents csv

GET /v1/user_agents/csv

Export agents to CSV.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agents csv
```

RESPONSE

HTTP 200 OK

Client Name,State,RC Version,RC Type,OS Name,Last Sync,Site Name,Policy/Schedule Name,Block Page Name

DESKTOP-01,protected,3.3.6,windows,Microsoft Windows 11 Pro
(10.0.26200.0),2026-05-21 17:22:17 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site

pixel-phone,protected,1.21.0,android,REL (14),2025-04-08 00:34:28 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site

DESKTOP-01,uninstalled,2.2.0,windows,Microsoft Windows 10 Pro (Microsoft Windows NT

10.0.19045.0),2025-08-13 14:03:58 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site
s22testfullvlock,uninstalled,1.21.1,android,REL (14),2025-04-20 16:40:11 -0400,Acme Accounting Co. HQ,Just block social media,Inherit from site
s22testfullvlock,uninstalled,1.21.1,android,REL (14),2025-04-19 16:16:57 -0400,Acme Accounting Co. HQ,Just block social media,Inherit from site
s22test,uninstalled,1.21.1,android,REL (14),2025-04-15 14:20:26 -0400,Acme Accounting Co. HQ,Block FB,Inherit from site
DESKTOP-01,uninstalled,2.2.0,windows,Microsoft Windows 10 Pro (Microsoft Windows NT 10.0.19045.0),2025-08-13 13:25:04 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site
desktop-prod2,uninstalled,2.0.8,windows,Microsoft Windows 10 Pro (Microsoft Windows NT 10.0.19045.0),2025-08-13 10:43:32 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site
sigofficeupd,protected,1.15.3.0,windows,Microsoft Windows 10 Pro (Microsoft Windows NT 10.0.19045.0),2025-09-29 11:15:21 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site
DESKTOP-01,uninstalled,3.3.2,windows,Microsoft Windows 11 Pro (10.0.26200.0),2026-03-24 08:52:00 -0400,Acme Accounting Co. HQ,Testing NXDomain blocks,Inherit from site
DESKTOP-01,protected,2.2.0,windows,Microsoft Windows 10 Pro (Microsoft Windows NT 10.0.19045.0),2025-08-13 13:14:43 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site
DESKTOP-01,protected,3.3.6,windows,Microsoft Windows 11 Pro (10.0.26200.0),2026-05-05 17:26:56 -0400,Acme Accounting Co. HQ,Inherit from site,Inherit from site

user-agents delete

DELETE /v1/user_agents/{id}

Permanently delete a user-agents resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agents delete --id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5
```

RESPONSE

HTTP 204 No Content

No response body — this is the expected success response.

HTTP 204 confirms the operation completed successfully.

user-agents dequeue-uninstall

POST /v1/user_agents/dequeue_uninstall

Dequeue a pending uninstall request for a specific agent..

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 285109

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py user-agents dequeue-uninstall
```

RESPONSE

HTTP 200 OK

```
{
  "status": "ok"
}
```

user-agents list

GET /v1/user_agents

Retrieve a paginated list of user-agents resources.

FLAGS

`--page` Page number for paginated results. Default: 1, e.g. `--page 1`
`--per_page` Number of results per page. Default: 25, e.g. `--per_page 25`
`--organization_ids` An array of organization IDs. Example: `["802315","802316"]`, e.g. `--organization_ids ["802315"]`
`--network_ids` Filter by network IDs (array), e.g. `--network_ids ["736401","736402"]`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--search` Filter results by a text search string, e.g. `--search "laptop"`
`--status` Filter by resource status, e.g. `--status active`
`--state` Filter by agent state, e.g. `--state "Colorado"`
`--tags` Agent tags as a JSON array. Example: `["managed","finance"]`, e.g. `--tags ["managed","finance"]`
`--sort` Sort field, e.g. `--sort "example"`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agents list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5",
      "type": "user_agents",
      "uuid": "0198a454-7812-77a6-8207-b96b1ffcab7b",
      "attributes": {
        "agent_type": "windows",
        "agent_version": "2.2.0",
        "agent_state": "protected",
        "client_id": "6866556666454026916776710065736752641527085667068467",
        "hostname": "DESKTOP-01",
        "status": "active",
        "status_code": null,
        "status_message": null,
        "os_version": "Microsoft Windows NT 10.0.19045.0",
        "os_name": "Microsoft Windows 10 Pro",
        "friendly_name": null,
        "tags": null,
        "current_user": {
          "login": "DESKTOP-01\\jsmith",
          "name": "jsmith",
          "last_update": "2025-08-13T12:47:41.744-04:00"
        },
        "created_at": "2025-08-13T12:47:35.954-04:00",
        "updated_at": "2026-03-24T08:40:59.628-04:00",
        "uninstalled_at": null,
        "uninstall_queued_at": null,
        "id_hex": "f161ffcfbdf654b8d5f273fd31fe3e7",
        "id_binary": [
          241,
          97,
          255,
          207,
          191,
          223,
          101,
          75,
          141,

```

```
95,  
39,  
63,  
211,  
31,  
227,  
231  
],  
"last_sync": "2025-08-13T13:14:43.539-04:00",  
"release_channel": "stable",  
"network_id": 736401,  
"network_name": "Acme Accounting Co. HQ",  
"cyber_sight_client": null,  
"cyber_sight_client_setting": null,  
"device_setting": null,  
"filtering_client_setting": null,  
"vpn_available": false,  
"vpn_status": null,  
"vpn_settings_user_agent": null  
},  
"relationships": {  
  ...
```

user-agents list-all

GET /v1/user_agents/all

Retrieve all user-agents resources without pagination limits.

EXAMPLE COMMAND

```
python dnsfcli.py user-agents list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5",
      "type": "user_agents",
      "uuid": "0198a454-7812-77a6-8207-b96b1ffcab7b",
      "attributes": {
        "agent_type": "windows",
        "agent_version": "2.2.0",
        "agent_state": null,
        "client_id": "6866556666454026916776710065736752641527085667068467",
        "hostname": "DESKTOP-01",
        "status": "active",
        "status_code": null,
        "status_message": null,
        "os_version": "Microsoft Windows NT 10.0.19045.0",
        "os_name": "Microsoft Windows 10 Pro",
        "friendly_name": null,
        "tags": null,
        "current_user": {
          "login": "DESKTOP-01\\jsmith",
          "name": "jsmith",
          "last_update": "2025-08-13T12:47:41.744-04:00"
        },
        "created_at": "2025-08-13T12:47:35.954-04:00",

```

```
"updated_at": "2026-03-24T08:40:59.628-04:00",
"uninstalled_at": null,
"uninstall_queued_at": null,
"id_hex": "f161ffcfbfdf654b8d5f273fd31fe3e7",
"id_binary": [
  241,
  97,
  255,
  207,
  191,
  223,
  101,
  75,
  141,
  95,
  39,
  63,
  211,
  31,
  227,
  231
],
"last_sync": "2025-08-13T13:14:43.539-04:00",
"release_channel": "stable",
"network_id": 736401,
"network_name": "Acme Accounting Co. HQ",
"cyber_sight_client": null,
"cyber_sight_client_setting": null,
"device_setting": null,
"filtering_client_setting": null,
"vpn_available": false,
"vpn_status": null,
"vpn_settings_user_agent": null
},
"relationships": {
  ...
```

user-agents show

GET /v1/user_agents/{id}

Retrieve a single user-agents resource by its ID.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py user-agents show --id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5",
    "type": "user_agents",
    "uuid": "0198a454-7812-77a6-8207-b96b1ffcab7b",
    "attributes": {
      "agent_type": "windows",
      "agent_version": "2.2.0",
```



```
"agent_state": "protected",
"client_id": "6866556666454026916776710065736752641527085667068467",
"hostname": "DESKTOP-01",
"status": "active",
"status_code": null,
"status_message": null,
"os_version": "Microsoft Windows NT 10.0.19045.0",
"os_name": "Microsoft Windows 10 Pro",
"friendly_name": null,
"tags": null,
"current_user": {
  "login": "DESKTOP-01\\jsmith",
  "name": "jsmith",
  "last_update": "2025-08-13T12:47:41.744-04:00"
},
"created_at": "2025-08-13T12:47:35.954-04:00",
"updated_at": "2026-03-24T08:40:59.628-04:00",
"uninstalled_at": null,
"uninstall_queued_at": null,
"id_hex": "f161ffcfbfd654b8d5f273fd31fe3e7",
"id_binary": [
  241,
  97,
  255,
  207,
  191,
  223,
  101,
  75,
  141,
  95,
  39,
  63,
  211,
  31,
  227,
  231
],
"last_sync": "2025-08-13T13:14:43.539-04:00",
"release_channel": "stable",
"network_id": 736401,
```

```
"network_name": "Acme Accounting Co. HQ",
"cyber_sight_client": null,
"cyber_sight_client_setting": null,
"device_setting": null,
"filtering_client_setting": null,
"vpn_available": false,
"vpn_status": null,
"vpn_settings_user_agent": null
},
"relationships": {
  "network": {
    ...
```

user-agents tags

GET /v1/user_agents/tags

List agent tags.

EXAMPLE COMMAND

```
python dnsfcli.py user-agents tags
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "tags": [
      "android"
    ]
  }
}
```

user-agents uninstall-pin

GET /v1/user_agents/uninstall_pin

Get uninstall PIN.

FLAGS

`--organization_id` [required] The numeric ID of the organization that owns this resource, e.g. `--organization_id 802315`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py user-agents uninstall-pin
```

RESPONSE

HTTP 200 OK

```
{
  "pin": "167757"
}
```

user-agents update

PATCH /v1/user_agents/{id}

Update an existing user-agents resource by its ID.

FLAGS

`--id` [required] The numeric ID of the resource to operate on, e.g. `--id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5`
`--friendly_name` The display name shown for this agent in the dashboard, e.g. `--friendly_name "Finance-Laptop"`
`--status` Filter by resource status, e.g. `--status active`
`--network_id` The numeric ID of the network to associate with, e.g. `--network_id 736401`
`--policy_id` The numeric ID of the filtering policy to assign, e.g. `--policy_id 285109`
`--scheduled_policy_id` The numeric ID of a time-based scheduled policy, e.g. `--scheduled_policy_id 71203`
`--block_page_id` The numeric ID of the block page to display when a domain is blocked, e.g. `--block_page_id 5147`
`--tags` Agent tags as a JSON array. Example: `["managed","finance"]`, e.g. `--tags '["managed","finance"]'`
`--vpn_settings_user_agent_attributes` VPN settings JSON, e.g. `--vpn_settings_user_agent_attributes {"key":"value"}`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnscli.py user-agents update \  
--id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5 \  
--friendly_name "Finance-Laptop" \  
--policy_id 285109 \  
--tags '["managed","finance"]'
```

RESPONSE

HTTP 200 OK

```
{  
  "id": "9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5",  
  "type": "user_agents",  
  "attributes": {  
    "friendly_name": "Finance-Laptop",  
    "policy_id": 285109,  
    "tags": [  
      "managed",  
      "finance"  
    ],  
    "updated_at": "2025-06-01T10:00:00.000-04:00"  
  }  
}
```

users change-password

PATCH /v1/users/change_password

Change current user password.

FLAGS

`--new_password [required]` The new password to set for the current user, e.g. `--new_password "NewSecurePass123!"`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py users change-password --new_password "NewSecurePass123!"
```

RESPONSE

HTTP 200 OK

```
{
  "status": "ok",
  "message": "Password changed successfully"
}
```

users list

GET /v1/users

Retrieve a paginated list of users resources.

FLAGS

--page Page number for paginated results. Default: 1, e.g. --page 1
--per_page Number of results per page. Default: 25, e.g. --per_page 25

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py users list --page 1 --per_page 25
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "120904",
      "type": "users",
      "uuid": "0186a050-d52c-7da0-b331-5de7b02fb5b7",
      "attributes": {
        "name": "Desktop C2",
```



```

    "email": "jdoe@example.com",
    "email_verified": false,
    "first_name": "Jane",
    "last_name": "C2",
    "phone": "",
    "mfa_enabled": false,
    "created_at": "2023-03-01T22:14:57.452-05:00",
    "created_at_epoch_utc": 1677726897,
    "updated_at": "2025-01-15T13:10:24.989-05:00",
    "intercom_user_verification":
"92b11adca8a14a0830437928c2d2f8406a0b77f997f03ecbd89c944d95f85179",
    "last_sign_in_at": "2024-12-18T10:41:32.902-05:00",
    "metadata": {},
    "must_reset_password": false
  }
},
{
  "id": "42618",
  "type": "users",
  "uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
  "attributes": {
    "name": "Jane Smith",
    "email": "jane.smith@example.com",
    "email_verified": true,
    "first_name": "Jane",
    "last_name": "Smith",
    "phone": "+14035128790",
    "mfa_enabled": true,
    "created_at": "2020-11-23T19:40:02.381-05:00",
    "created_at_epoch_utc": 1606178402,
    "updated_at": "2026-05-21T17:21:26.365-04:00",
    "intercom_user_verification":
"b480ce6135d61bb46b4fe92465d7dec45295c9880f07fff6759ad4ca575edeee",
    "last_sign_in_at": "2026-05-21T17:21:26.365-04:00",
    "metadata": {},
    "must_reset_password": false
  }
}
],
"links": {
  "self": "https://api.dnsfilter.com/v1/users?

```

```
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",  
  "first": "https://api.dnsfilter.com/v1/users?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2",  
  "prev": null,  
  "next": null,  
  "last": "https://api.dnsfilter.com/v1/users?  
page%5Bnumber%5D=1&page%5Bsize%5D=1500&per_page=2"  
}  
}
```

users list-all

GET /v1/users/all

Retrieve all users resources without pagination limits.

EXAMPLE COMMAND

```
python dnscli.py users list-all
```

RESPONSE

HTTP 200 OK

```
{
  "data": [
    {
      "id": "42618",
      "type": "users",
      "uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
      "attributes": {
        "name": "Jane Smith",
        "email": "jane.smith@example.com",
        "email_verified": true,
        "first_name": "Jane",
        "last_name": "Smith",
        "phone": "+14035128790",
        "mfa_enabled": true,
        "created_at": "2020-11-23T19:40:02.381-05:00",
        "created_at_epoch_utc": 1606178402,
        "updated_at": "2026-05-21T17:21:26.365-04:00",
        "intercom_user_verification":
          "b480ce6135d61bb46b4fe92465d7dec45295c9880f07fff6759ad4ca575edeee",
        "last_sign_in_at": "2026-05-21T17:21:26.365-04:00",
        "metadata": {},
        "must_reset_password": false
      }
    },
    {
```

```

    "id": "120904",
    "type": "users",
    "uuid": "0186a050-d52c-7da0-b331-5de7b02fb5b7",
    "attributes": {
      "name": "Desktop C2",
      "email": "jdoe@example.com",
      "email_verified": false,
      "first_name": "Jane",
      "last_name": "C2",
      "phone": "",
      "mfa_enabled": false,
      "created_at": "2023-03-01T22:14:57.452-05:00",
      "created_at_epoch_utc": 1677726897,
      "updated_at": "2025-01-15T13:10:24.989-05:00",
      "intercom_user_verification":
"92b11adca8a14a0830437928c2d2f8406a0b77f997f03ecbd89c944d95f85179",
      "last_sign_in_at": "2024-12-18T10:41:32.902-05:00",
      "metadata": {},
      "must_reset_password": false
    }
  },
  "links": {
    "self": "https://api.dnsfilter.com/v1/users/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "first": "https://api.dnsfilter.com/v1/users/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500",
    "prev": null,
    "next": null,
    "last": "https://api.dnsfilter.com/v1/users/all?page%5Bnumber%5D=1&page%5Bsize%5D=1500"
  }
}

```

users show

GET /v1/users/{id}

Retrieve a single users resource by its ID.

FLAGS

`--id [required]` The numeric ID of the resource to operate on, e.g. `--id 42618`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py users show --id 42618
```

RESPONSE

HTTP 200 OK

```
{
  "data": {
    "id": "42618",
    "type": "users",
    "uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
    "attributes": {
      "name": "Jane Smith",
      "email": "jane.smith@example.com",
      "email_verified": true,
```

```
"first_name": "Jane",
"last_name": "Smith",
"phone": "+14035128790",
"mfa_enabled": true,
"created_at": "2020-11-23T19:40:02.381-05:00",
"created_at_epoch_utc": 1606178402,
"updated_at": "2026-05-21T17:21:26.365-04:00",
"intercom_user_verification":
"b480ce6135d61bb46b4fe92465d7dec45295c9880f07fff6759ad4ca575edeee",
"last_sign_in_at": "2026-05-21T17:21:26.365-04:00",
"metadata": {},
"must_reset_password": false
}
}
}
```

v2-agent-local-users counts

GET /v2/agent_local_users/counts

Count agent local users.

EXAMPLE COMMAND

```
python dnsfcli.py v2-agent-local-users counts
```

RESPONSE

HTTP 200 OK

```
{  
  "all": 4,  
  "user_policy_override": 1,  
  "inherited_policy": 3  
}
```

v2-agent-local-users csv-export

POST /v2/agent_local_users_csv_export

Create agent local users CSV export.

FLAGS

`--organization_ids` An array of organization IDs. Example: `["802315","802316"]`, e.g. `--organization_ids '["802315"]'`
`--name` A human-readable display name for this resource, e.g. `--name "Guest WiFi"`
`--search` Filter results by a text search string, e.g. `--search "laptop"`
`--user_policy_override` Filter by policy override, e.g. `--user_policy_override false`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py v2-agent-local-users csv-export --organization_ids '["802315"]'
```

RESPONSE

HTTP 202 Accepted

```
{
  "id": "alu-export-001",
  "status": "pending",
```



```
"created_at": "2025-06-01T10:00:00.000-04:00"  
}
```

v2-agent-local-users csv-export-show

GET /v2/agent_local_users_csv_export/{id}

Show a CSV export job.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id alu-export-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py v2-agent-local-users csv-export-show --id alu-export-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "alu-export-001",
  "status": "completed",
  "download_url": "https://api.dnsfilter.com/exports/alu-export-001.csv",
  "completed_at": "2025-06-01T10:00:12.000-04:00"
}
```

v2-current-user suppress-license-warning

POST /v2/current_user/suppress_license_warning

Suppress the license warning for the current user..

FLAGS

`--organization_uuid` UUID of the organization to suppress the warning for, e.g. `--organization_uuid "example"`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`

`--json` Output JSON to stdout (automatic when piped), e.g. `--json`

`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`

`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`

`--template` Print a blank CSV import template for this command and exit, e.g. `--template`

`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`

`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`

`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py v2-current-user suppress-license-warning
```

RESPONSE

HTTP 200 OK

```
{
  "status": "ok"
}
```

v2-current-user ui-settings

GET /v2/current_user/ui_settings

Show UI settings.

EXAMPLE COMMAND

```
python dnsfcli.py v2-current-user ui-settings
```

RESPONSE

HTTP 200 OK

```
{
  "user_uuid": "0175f7b1-704d-72f8-9855-538ed456663c",
  "disable_license_warnings": false,
  "theme_mode": "system"
}
```

v2-current-user ui-settings-update

PATCH /v2/current_user/ui_settings

Update UI settings.

FLAGS

`--disable_license_warnings` Disable license warnings, e.g. `--disable_license_warnings false`
`--user_uuid` User UUID, e.g. `--user_uuid 0175f7b1-704d-72f8-9855-538ed456663c`
`--theme_mode` UI theme. One of: light, dark, system, e.g. `--theme_mode dark`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py v2-current-user ui-settings-update --theme_mode dark
```

RESPONSE

HTTP 200 OK

```
{
  "theme_mode": "dark",
  "disable_license_warnings": false,
  "updated_at": "2025-06-01T10:00:00.000-04:00"
}
```

v2-dictionary cyber-sight-activity-types

GET /v2/dictionary/cyber_sight_activity_types

List Cyber Sight activity types.

EXAMPLE COMMAND

```
python dnsfcli.py v2-dictionary cyber-sight-activity-types
```

RESPONSE

HTTP 200 OK

```
[
  {
    "id": 2,
    "name": "idle",
    "friendly_name": "Idle",
    "description": "The user has not interacted with the system for over two minutes."
  },
  {
    "id": 3,
    "name": "lock",
    "friendly_name": "Machine Lock",
    "description": "The user has locked their machine."
  },
  {
    "...": "(6 more)"
  }
]
```

v2-dictionary vpn-settings-state-types

GET /v2/dictionary/vpn_settings_state_types

List VPN settings state types.

EXAMPLE COMMAND

```
python dnscli.py v2-dictionary vpn-settings-state-types
```

RESPONSE

HTTP 200 OK

```
[
  {
    "id": 1,
    "name": "manual",
    "friendly_name": "Manual",
    "description": "The VPN is currently set to allow manual connections."
  },
  {
    "id": 2,
    "name": "always-on",
    "friendly_name": "Always On",
    "description": "The VPN is currently set to always be on."
  },
  {
    "...": "(1 more)"
  }
]
```

v2-networks csv-export

POST /v2/networks_csv_export

Create networks CSV export.

FLAGS

`--organization_ids` An array of organization IDs. Example: `["802315","802316"]`, e.g. `--organization_ids '["802315"]'`
`--msp_id` MSP ID, e.g. `--msp_id 8801`
`--ids` An array of resource IDs. Example: `["id1","id2"]`, e.g. `--ids ["9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"]`

Common global flags (every command; see the Global Flags page for all of them):

`--raw` Return raw JSON instead of the formatted table, e.g. `--raw`
`--json` Output JSON to stdout (automatic when piped), e.g. `--json`
`--to-csv FILE` Write the response to a CSV file, e.g. `--to-csv output.csv`
`--from-csv FILE` Read input rows from a CSV file (one API call per row), e.g. `--from-csv policies.csv`
`--template` Print a blank CSV import template for this command and exit, e.g. `--template`
`--filter EXPR` Keep matching rows only, e.g. `--filter status=active`
`--columns a,b,c` Limit table output to named columns, e.g. `--columns id,name`
`--org-id ID` Override the stored organization ID for this call only, e.g. `--org-id 802315`

EXAMPLE COMMAND

```
python dnsfcli.py v2-networks csv-export --organization_ids '["802315"]'
```

RESPONSE

HTTP 202 Accepted

```
{
  "id": "net-export-001",
  "status": "pending",
```



```
"created_at": "2025-06-01T10:00:00.000-04:00"  
}
```

v2-networks csv-export-show

GET /v2/networks_csv_export/{id}

Show a network export job.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id net-export-001

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw

--json Output JSON to stdout (automatic when piped), e.g. --json

--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv

--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv

--template Print a blank CSV import template for this command and exit, e.g. --template

--filter EXPR Keep matching rows only, e.g. --filter status=active

--columns a,b,c Limit table output to named columns, e.g. --columns id,name

--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py v2-networks csv-export-show --id net-export-001
```

RESPONSE

HTTP 200 OK

```
{
  "id": "net-export-001",
  "status": "completed",
  "download_url": "https://api.dnsfilter.com/exports/net-export-001.csv",
  "completed_at": "2025-06-01T10:00:10.000-04:00"
}
```

v2-user-agents update-settings

PATCH /v2/user_agents/{id}/update_settings

Update agent settings.

FLAGS

--id [required] The numeric ID of the resource to operate on, e.g. --id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5
--device_setting_attributes Device settings JSON, e.g. --device_setting_attributes {"auto_update":true}
--filtering_client_setting_attributes Filter settings JSON, e.g. --filtering_client_setting_attributes {"block_malware":true}

Common global flags (every command; see the Global Flags page for all of them):

--raw Return raw JSON instead of the formatted table, e.g. --raw
--json Output JSON to stdout (automatic when piped), e.g. --json
--to-csv FILE Write the response to a CSV file, e.g. --to-csv output.csv
--from-csv FILE Read input rows from a CSV file (one API call per row), e.g. --from-csv policies.csv
--template Print a blank CSV import template for this command and exit, e.g. --template
--filter EXPR Keep matching rows only, e.g. --filter status=active
--columns a,b,c Limit table output to named columns, e.g. --columns id,name
--org-id ID Override the stored organization ID for this call only, e.g. --org-id 802315

EXAMPLE COMMAND

```
python dnsfcli.py v2-user-agents update-settings --id 9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5
```

RESPONSE

HTTP 200 OK

```
{  
  "id": "9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5",  
  "status": "ok",
```

```
"updated_at": "2025-06-01T10:00:00.000-04:00"  
}
```

From the team at

DNSFilter